Computer Networks

중간 정리

중간고사 공지

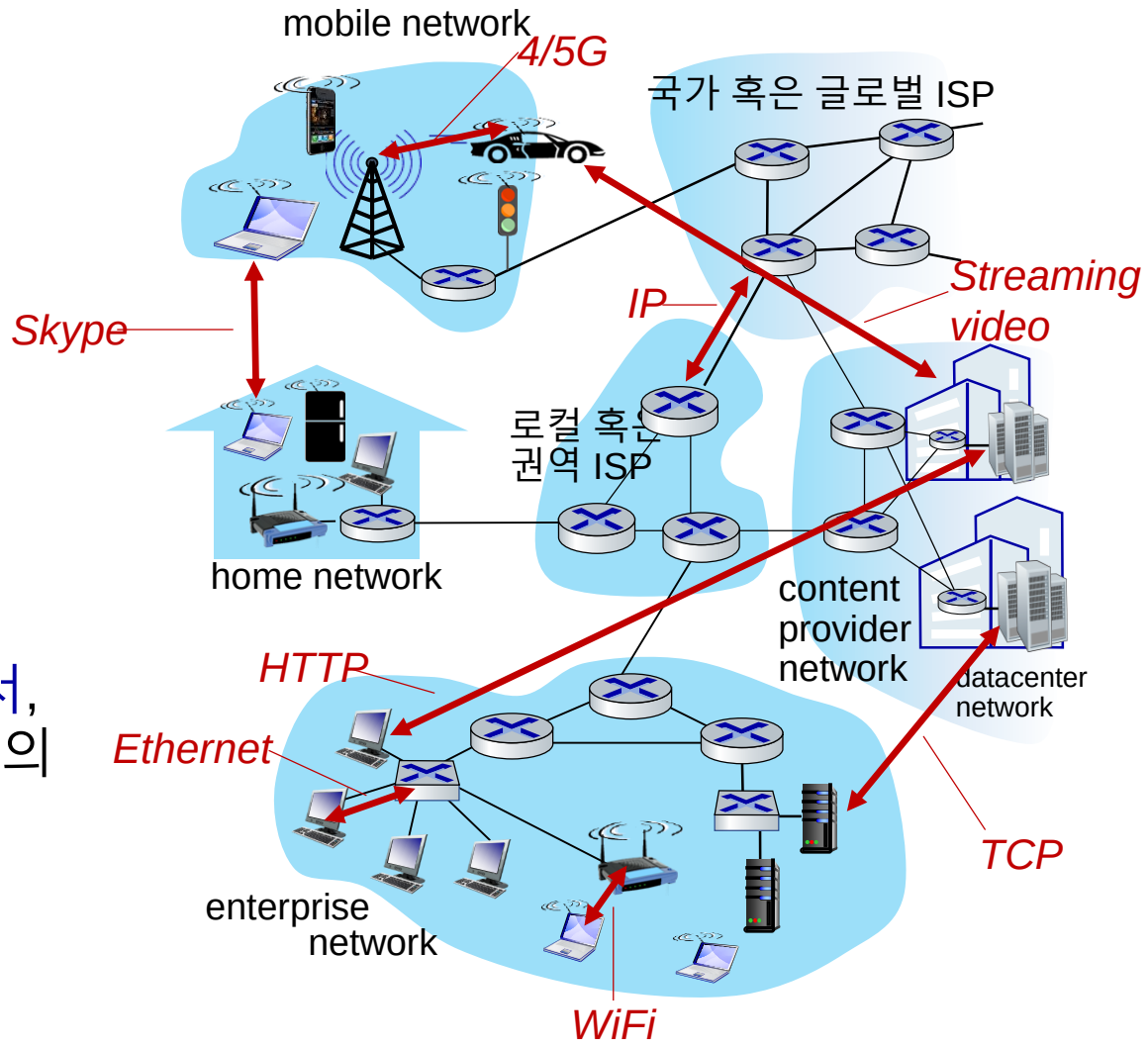
- 일정: 4월 24일 (금) 오전 10:00 ~ 11:30
  - D128호: 컴퓨터공학전공2분반 및 인공지능공학전공
  - D129호: 컴퓨터공학전공1분반
  - 시험 시간이 중복된 학생은 4월 22일 (수)까지 네이버웍스 또는 메일(anten@kumoh.ac.kr)로 반드시 보고 → 4월 24일 (금) 오전 8:30부터 D139호에서 중간고사를 치를 예정
- 범위: 1 ~ 3장
- 중간고사 팁
  - [TMI]는 시험에 출제 되지 않음
  - 중간 정리 수업을 바탕으로 중간고사를 준비하면 효율적
  - 강의 노트의 예제, 과제, 판서 내용을 기본적으로 숙지
  - 부분 점수를 위해서 계산 과정을 남기고, 반드시 검산 (계산기 사용 가능)
  - 단위 환산 신경 쓰고, 단위를 반드시 표시
- 성적은 시험지 번호로 공지: 시험지 첫 페이지 상단의 시험지 번호를 반드시 암기

1. Computer Networks and the Internet

- What is the Internet? What is a protocol?
- Network edge: hosts, access network, physical media
- Network core: packet/circuit switching, internet structure
- Performance: loss, delay, throughput
- Protocol layers, service models
- Security
- History

구성요소로 본 인터넷

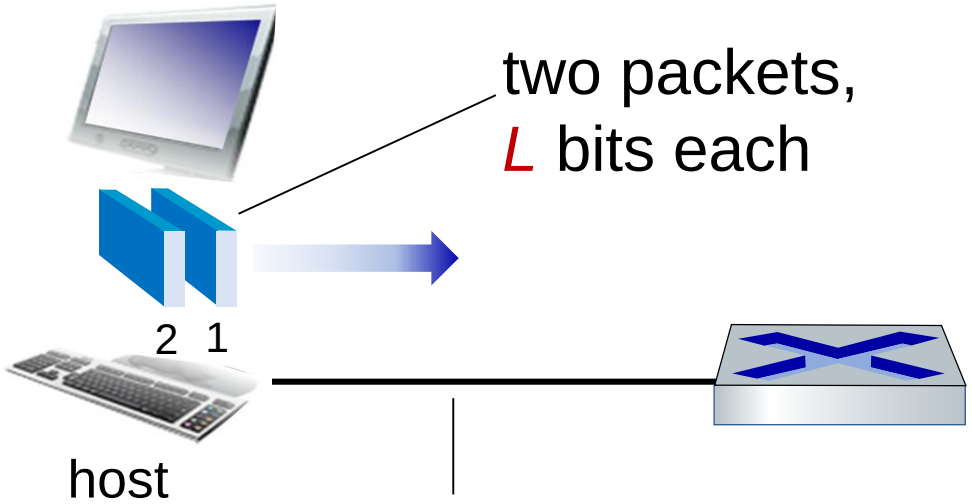
- Internet: ‘network of networks’
  - ISP들은 인터넷을 통해 상호 연결됨
- 인터넷의 구성요소
  - 호스트, 패킷 스위치, 통신 링크, 네트워크
- 호스트 (종단 시스템)
  - running network apps at Internet’s ‘edge’ (가장자리)
- 프로토콜 (protocols) :
  - 통신 개체 간에 송수신되는 메시지의 형식과 순서, 그리고 메시지 전송 및 수신 시 수행할 동작을 정의



Host: 데이터 패킷 전송

호스트의 전송 기능:

- 분산 애플리케이션의 메시지를 *L* 비트의 패킷(packets)으로 나눔
  - 일반적으로 *L* 비트는 최대 전송 단위 (maximum transmission unit, *MTU*)
- 패킷을 전송 속도 *R*로 접속 네트워크로 전송
- 패킷 전송 지연 (transmission delay): R bps의 링크에 *L*-bit 패킷을 전송 (밀어 넣기)하는 데 *L/R*초가 걸림
  - 출발지와 목적지 사이에 *N*개의 링크가 있다면 종단 간 전송 지연 시간은 약 *N·L/R*



R: link transmission rate

$$\text{패킷 전송 지연} = \frac{L\ (\text{bits})}{R\ (\text{bits/sec})}$$

A closer look at Internet structure

Network edge:

- hosts: clients and servers
- servers often in data centers

Access networks (접속 네트워크):

- 호스트와 경로상의 첫 번째 라우터 (엣지 라우터) 간의 연결을 담당
  - Home access networks
  - Mobile access networks (WiFi, 4G/5G)
  - Enterprise access networks (school, company)
  - Data center access networks

Physical media (물리 매체): wired, wireless communication links

Network core

Network core:

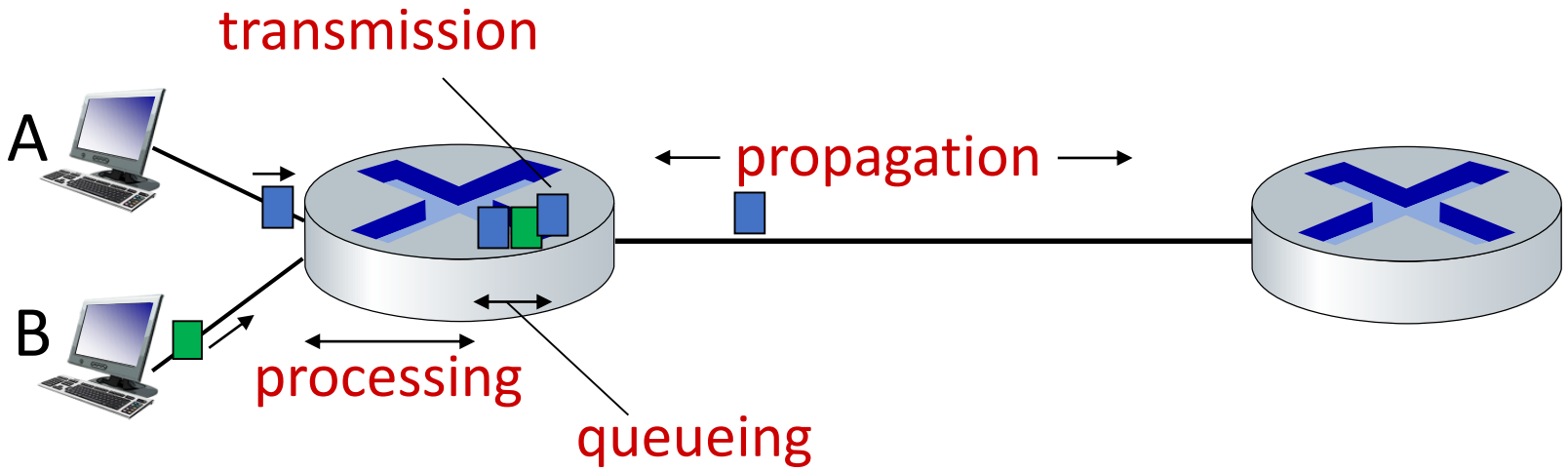
- 상호 연결된 라우터들의 그물망
- Packet-switching (패킷 교환) 방식
- Circuit switching (회선 교환) 방식

저장-후-전달 (store and forward):

- 하나의 패킷이 라우터에 완전히 도착해야 다음 링크로 전송할 수 있음

현재 인터넷 구조는 경제적 요인 (economics)과 국가 정책 (national policies)에 의해 진화됨

패킷 지연: 4개 원인



$$d_{packet} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$$

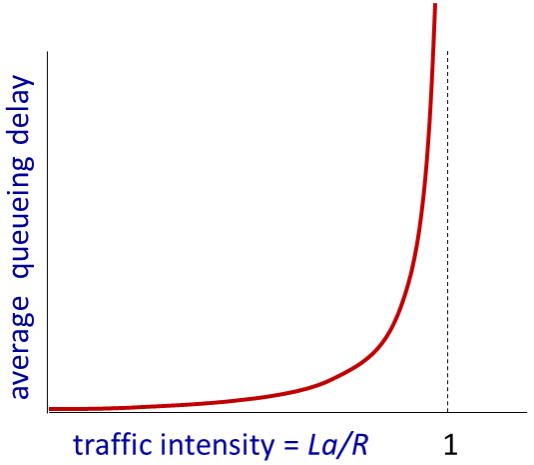
d_{proc}: 처리 지연 (processing delay)
d_{queue}: 큐잉 지연 (queueing delay)
d_{trans}: 전송 지연 (transmission delay)
d_{prop}: 전파 지연 (propagation delay)

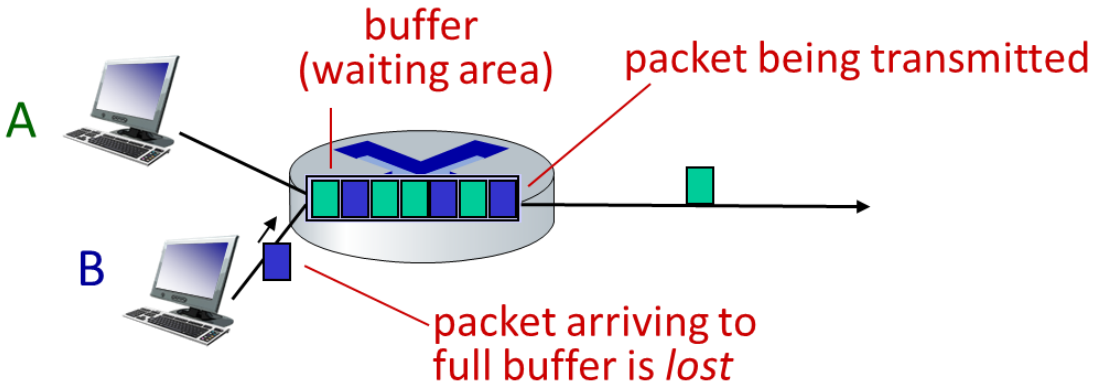
트래픽 강도 및 패킷 손실

- 트래픽 강도 (traffic intensity):
  - La/R이 0에 근접: 평균 큐잉 지연이 작음
  - La/R이 1에 근접: 평균 큐잉 지연이 큼

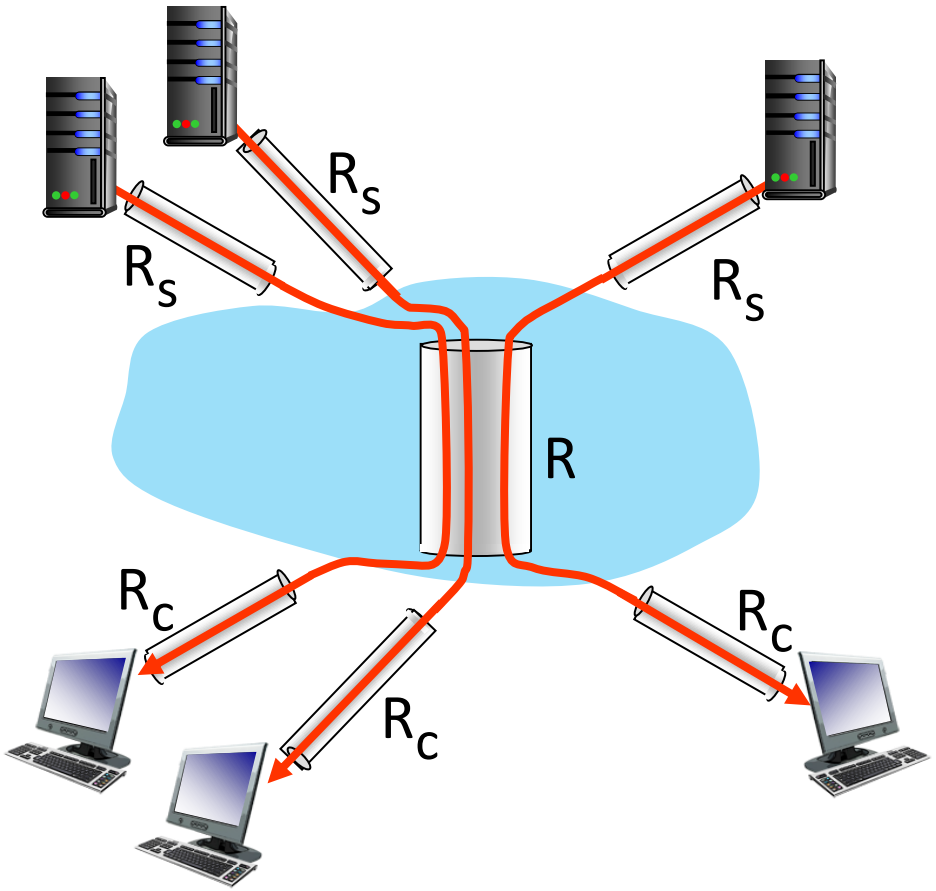
$$\frac{L \cdot a}{R} : \frac{\text{arrival rate of bits}}{\text{service rate of bits}}$$

- 패킷 손실 비율 (packet loss ratio): 전체 전송된 패킷 중 손실된 패킷의 비율
  - 큐(버퍼) 용량은 한계가 있음
  - packet arriving to full queue dropped (lost)
  - lost packet may be retransmitted by previous node, by source host, or not at all





처리율



10 connections (fairly) share backbone bottleneck link *R* bits/sec

- 처리율 (Throughput): 단위 시간 (time unit) 당 출발지로부터 목적지까지 전달 된 비트의 양 (bits/sec)
  - 순간적 처리율
  - 평균 처리율

병목 링크

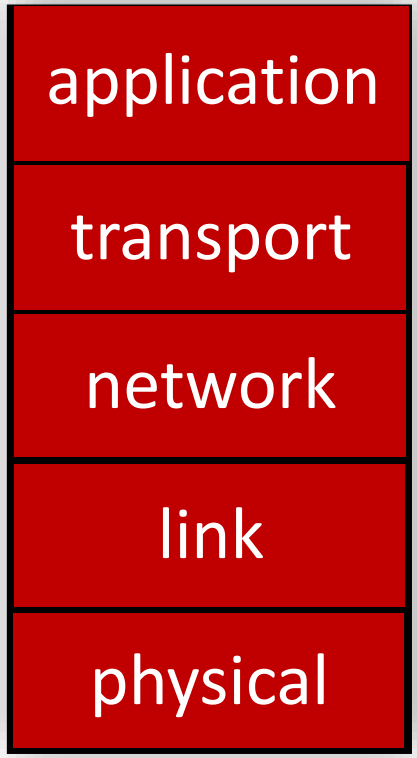
종단 간 경로 상에서 종단 간 처리율을 제한시키는 링크

계층화와 캡슐화

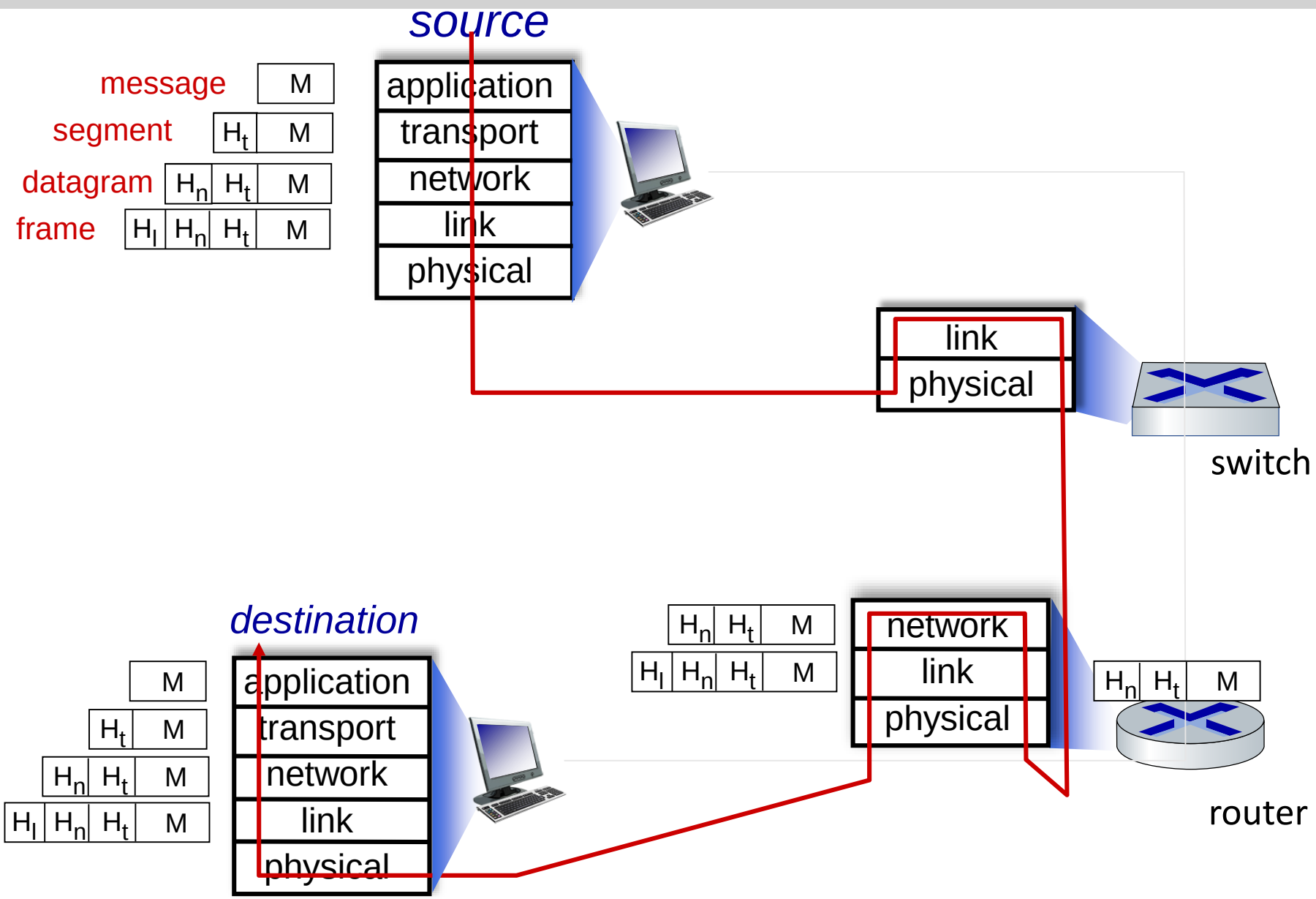
- 계층화: 복잡한 시스템을 설계하는 접근 방식
  - 명시적 구조로 시스템 구성 요소의 식별 및 관계 파악 가능
- 각 계층의 서비스를 모듈화로 유지보수 및 시스템 업데이트가 용이
  - 특정 계층의 서비스 구현 변경: 다른 계층에 투명
- 캡슐화 (Encapsulation)
  - header + payload (상위 객체의 패킷)

Layered Internet protocol stack (프로토콜 스택)

- application: supporting network applications
  - HTTP, IMAP, SMTP, DNS
- transport: process-process data transfer
  - TCP, UDP
- network: routing of datagrams from source to destination
  - IP, routing protocols
- link: data transfer between neighboring network elements
  - Ethernet, 802.11 (WiFi)
- physical: bits ‘on the wire’



Encapsulation: an end-end view



2. Application layer

- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- Domain Name System, DNS
- P2P applications
- Video streaming and content distribution networks
- Socket programming with UDP and TCP

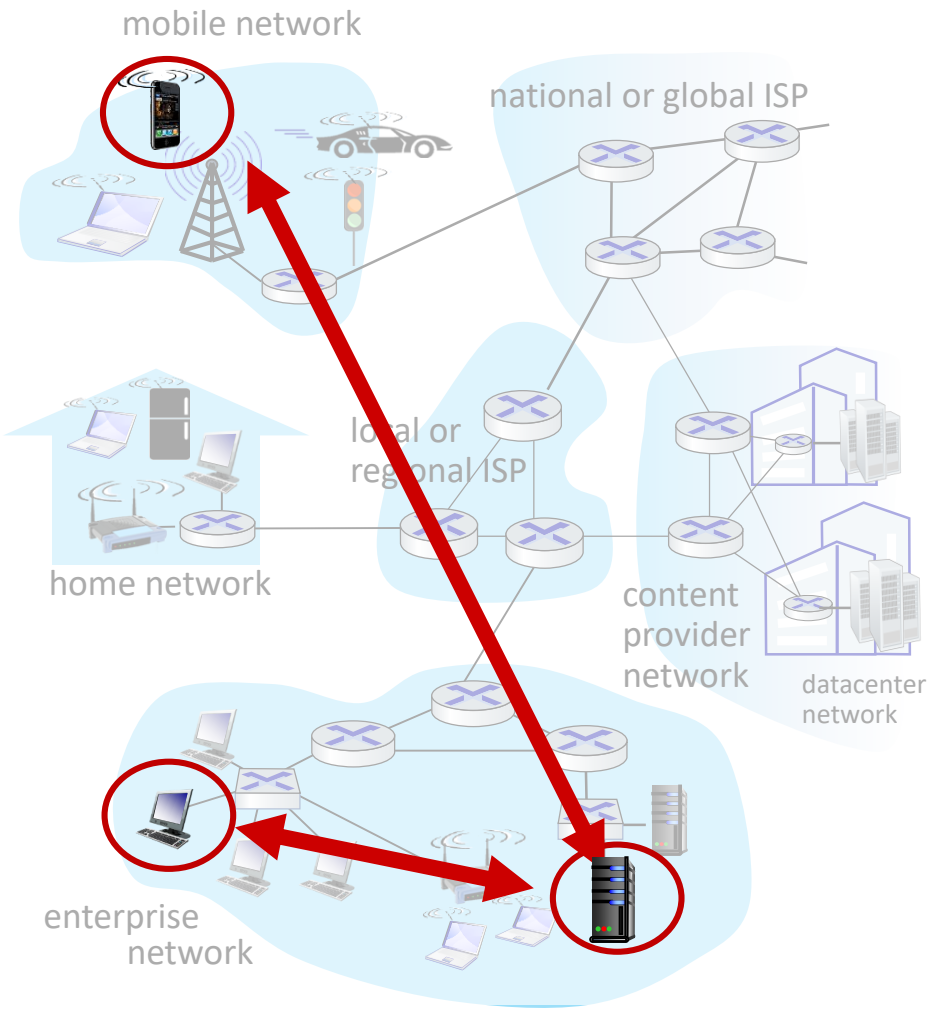
Client-server architecture

server:

- 항상 동작하는 호스트
- IP 주소가 고정
- 확장성 때문에 데이터 센터에 있는 경우가 많음

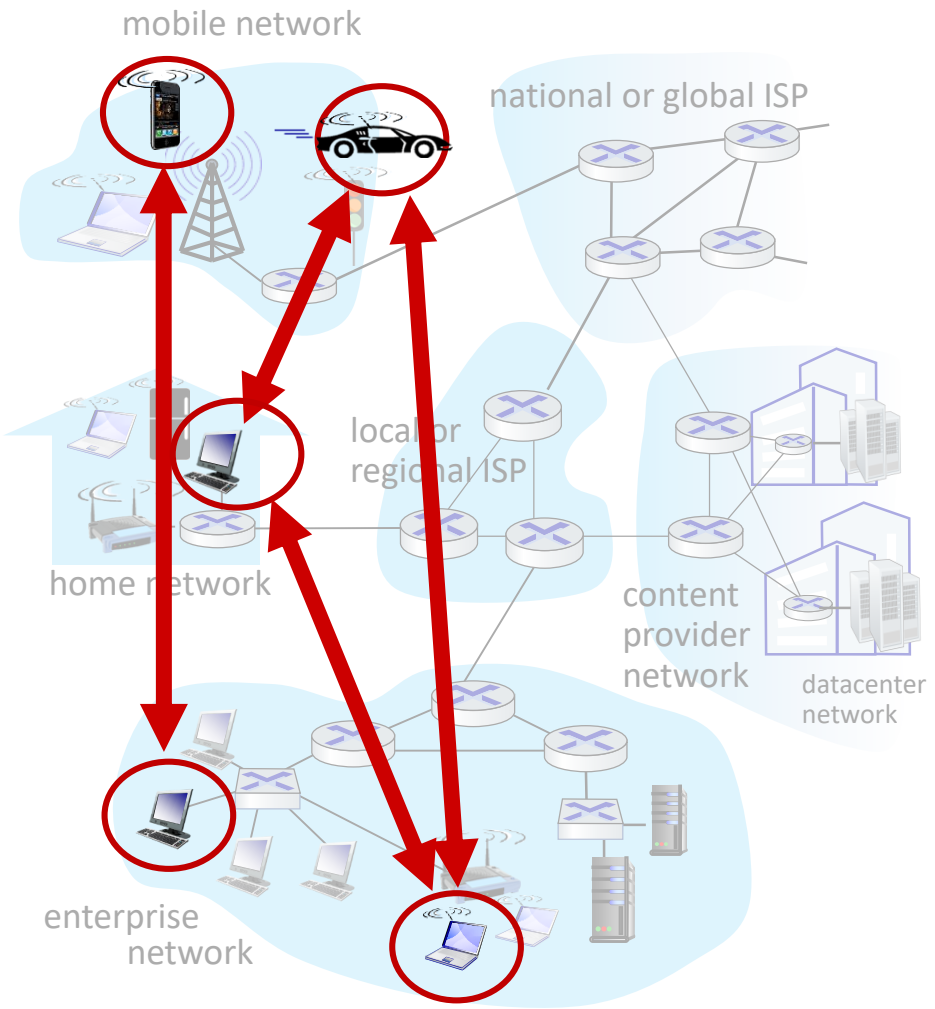
clients:

- 서버와 접속 및 통신
- 서버와 가끔씩 연결
- IP 주소가 변경됨
- 클라이언트 간의 ‘직접’ 통신은 하지 않음
- [예] HTTP, IMAP, FTP



Peer-to-peer architecture

- 항상 동작하는 서버가 없음
- 임의의 호스트 간의 직접 통신
- 피어 (peer)는 다른 피어들에게 서비스 ‘요청’과 서비스 ‘제공’을 동시에 수행
  - 자기 확장성 (self scalability): 새로운 피어는 새로운 서비스를 요구하지만 새로운 서비스 용량도 제공
- 피어가 간헐적으로 연결되고 IP 주소가 변경될 수 있음
  - 고도의 분산 구조로 보안, 성능, 신뢰성 측면에서 관리하기 복잡함
- [예] P2P file sharing (BitTorrent), Skype



애플리케이션은 어떤 트랜스포트 서비스가 필요?

① 신뢰적 데이터 전송 (reliable data transfer)

- 일부 앱 ([예] 파일 전송, 웹 트랜잭션)은 100% 안정적인 데이터 전송이 필요
- 다른 앱 ([예] 오디오)은 약간의 손실을 허용할 수 있음: 손실 허용 (loss-tolerant) 앱

② 시간 보장 (timing guarantee)

- 일부 앱 ([예] 인터넷 전화, 실시간 상호작용형 게임)은 '효과적'으로 동작하려면 지연 시간이 짧아야 함

③ 처리율 (throughput)

- 일부 앱 ([예] 멀티미디어)은 '효과적'으로 동작하기 위해 최소한의 처리율이 필요: 대역폭 민감 (bandwidth-sensitive) 앱
- 다른 앱은 주어진 처리율을 활용: 탄력적 (elastic) 앱

④ 보안 (security)

- encryption, data integrity, …

Internet transport protocols services

TCP (Transmission Control Protocol) 서비스:

- 신뢰적 데이터 전송 (reliable transport): between sending and receiving process
- 흐름 제어 (flow control): sender won't overwhelm receiver
- 혼잡 제어 (congestion control): control sender when network overloaded
- 연결 지향형 (connection-oriented): setup required between client and server processes
- *does not provide*: timing, minimum throughput guarantee, security

UDP (User Datagram Protocol) 서비스:

- *unreliable data transfer*: between sending and receiving process
- *does not provide*: reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup
- 'best-effort' IP의 단순한 확장

Q: UDP를 사용하는 이유는?

Securing TCP

일반 TCP & UDP 소켓:

- 암호화 없음
- 소켓을 통해 전송된 평문 (plain text) 비밀번호가 인터넷을 그대로 통과

Transport Layer Security (TLS)

- 전송 계층 (transport layer) 위에서 동작하는 널리 사용되는 보안 프로토콜
  - 거의 모든 웹 브라우저 및 웹 서버에서 지원: https (port 443)
- 주요 기능
  - 암호화된 TCP 연결 제공
  - 데이터 무결성 보장
  - 종단 간 (end-point) 인증 지원

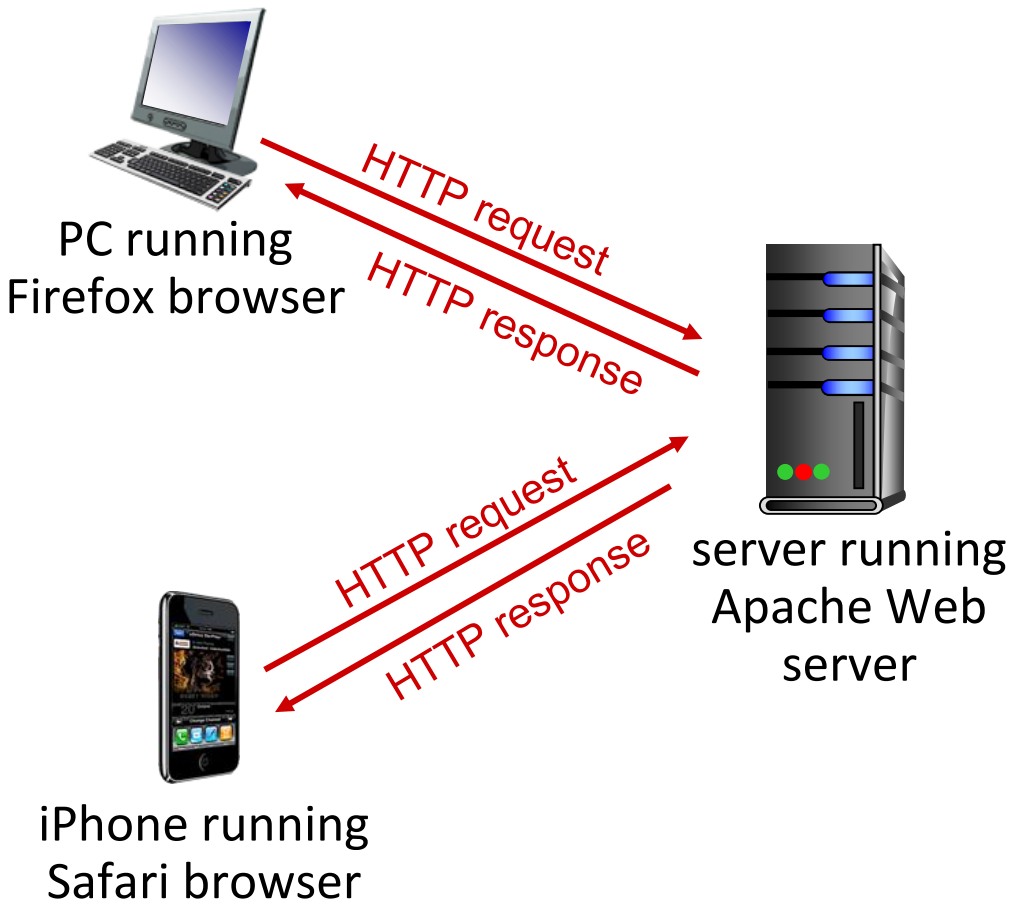
TLS는 애플리케이션 계층에서 구현됨

- 애플리케이션은 TLS 라이브러리를 사용하며, TLS는 TCP 위에서 동작
- 소켓에 입력된 평문 데이터는 암호화된 형태로 인터넷을 통과

HTTP overview

Web

- 웹 페이지는 각각 다른 웹 서버에 저장될 수 있는 객체들 (objects)로 구성
- 웹 페이지는 여러 참조 객체 (referenced object)를 포함하는 base HTML 파일로 구성되며, 각 객체는 URL (uniform resource locator)로 지정할 수 있음
- client/server model:
  - client: browser that requests, receives, (using HTTP protocol) and 'displays' Web objects
  - server: Web server sends (using HTTP protocol) objects in response to requests



HTTP connections: two types

비지속 연결 (non-persistent connection) HTTP

- ① TCP connection opened
- ② at most one object sent over TCP connection
- ③ TCP connection closed

비지속 연결 (HTTP1)의 단점들:

- ① requires 2 RTTs per object → 병렬 TCP 연결
- ② OS overhead for each TCP connection

병렬 TCP 연결 사용의 동기가 많음

지속 연결 (persistent connection) HTTP

- TCP connection opened to a server
- multiple objects can be sent over *single* TCP connection between client, and that server
- TCP connection closed
- 모든 참조된 객체에 대해 단 한 번의 RTT만 필요 (응답 시간이 절반으로 단축)
- 응답을 기다리지 않고 연속적으로 요청하는 파이프라이닝 기법 활용

HTTP request message

- two types of HTTP messages: *request, response*
- HTTP request message:
  - ASCII (human-readable format)

carriage return character

line-feed character

request line (GET, POST, PUT, DELETE, HEAD commands)

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X
   10.15; rv:80.0) Gecko/20100101 Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
\r\n
```

header lines

carriage return, line feed at start of line indicates end of header lines

HTTP response message

상태 라인 (status line) : protocol, status code, status phrase

header lines

data, [예] requested HTML file

```
HTTP/1.1 200 OK
Date: Tue, 08 Sep 2020 00:53:20 GMT
Server: Apache/2.4.6 (CentOS)
   OpenSSL/1.0.2k-fips PHP/7.4.9
   mod_perl/2.0.11 Perl/v5.16.3
Last-Modified: Tue, 01 Mar 2016 18:57:50 GMT
Accept-Ranges: bytes
Content-Length: 2651
Content-Type: text/html; charset=UTF-8
\r\n
data data data data data ...
```

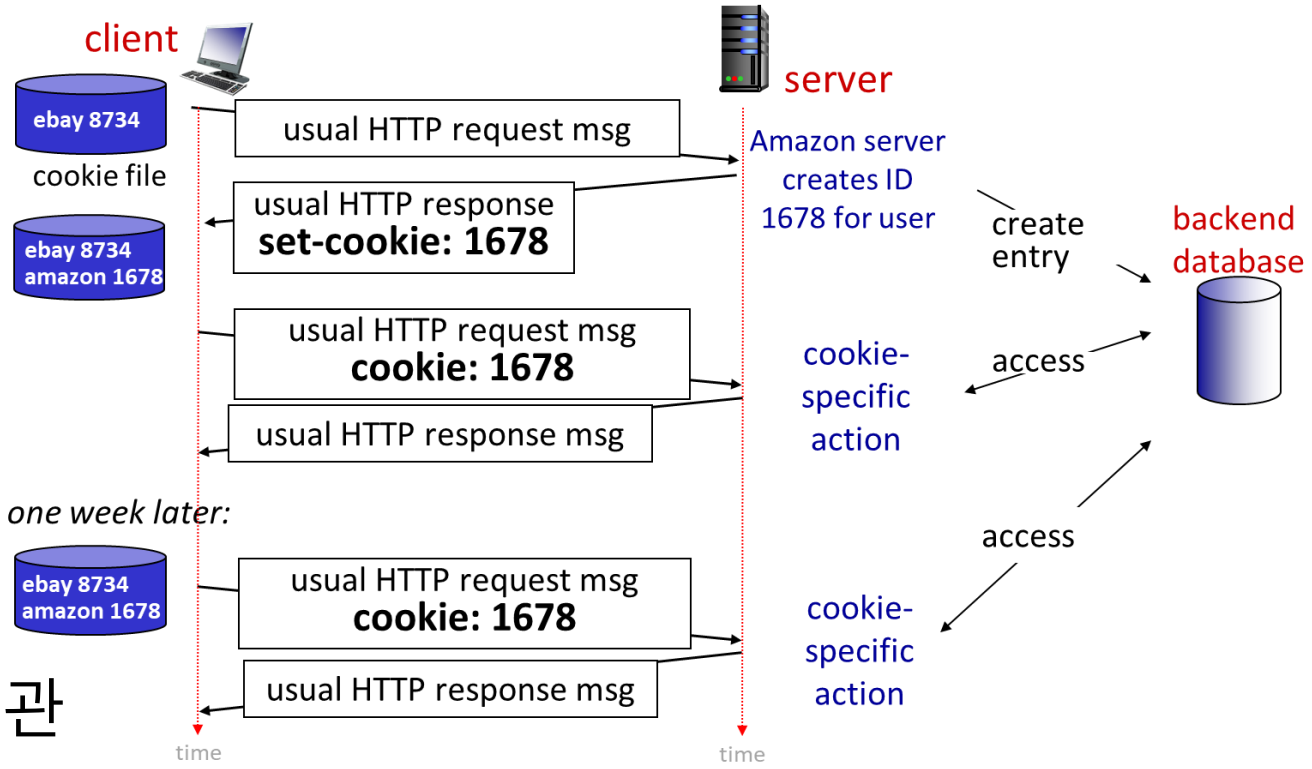
Maintaining user/server state: cookies

Recall: HTTP is *stateless*

- how to auto-login on website?
- Web sites and client browser use cookies to maintain some state between transactions

four components:

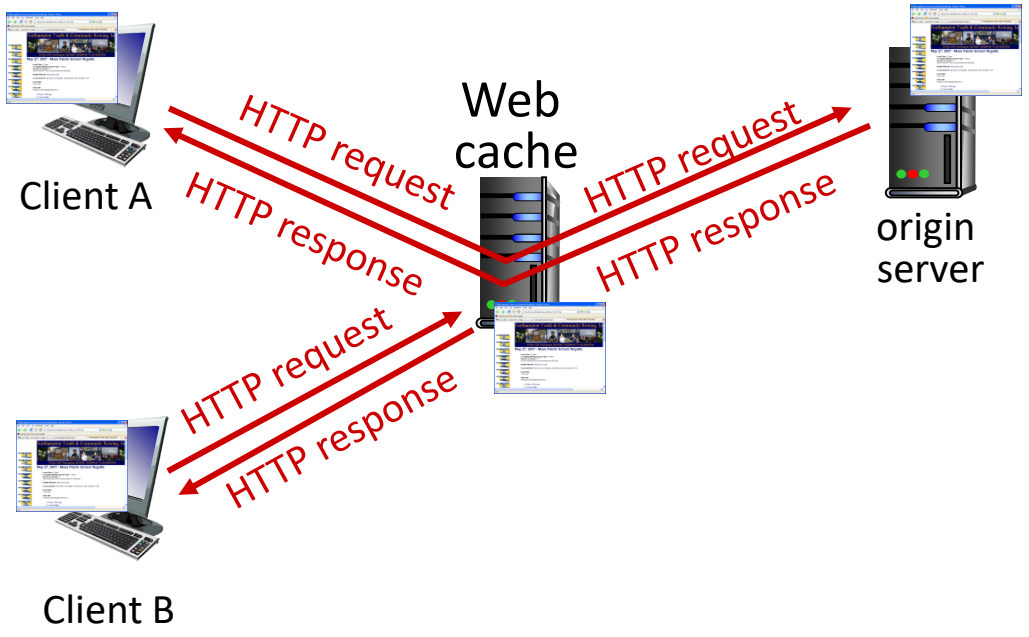
- ① HTTP 요청 메시지 쿠키 헤더 라인
- ② HTTP 응답 메시지 쿠키 헤더 라인
- ③ 쿠키 파일을 사용자의 호스트에 보관 및 사용자의 브라우저에서 관리
- ④웹사이트의 백엔드(back-end) 데이터베이스



Web caches (proxy servers)

Goal: 기점 서버 (origin server)의 개입 없이 클라이언트의 요청을 충족

- 웹 캐시 (*Web cache*)는 최근 요청된 객체의 사본을 저장 및 보존
- 브라우저는 사용자의 모든 HTTP 요청을 웹 캐시에 가장 먼저 전송되도록 구성
  - *if* object in cache: cache returns object to client
  - *else* cache requests object from origin server, caches received object, then returns object to client



조건부 GET

problem: 캐시된 객체의 복사본이 최신이 아닐 수 있음 (즉, 기점 서버의 객체가 갱신됨)

- 조건부 (conditional) GET
  - *client*: specify date of cached copy in HTTP request via GET method

    If-modified-since: <date>

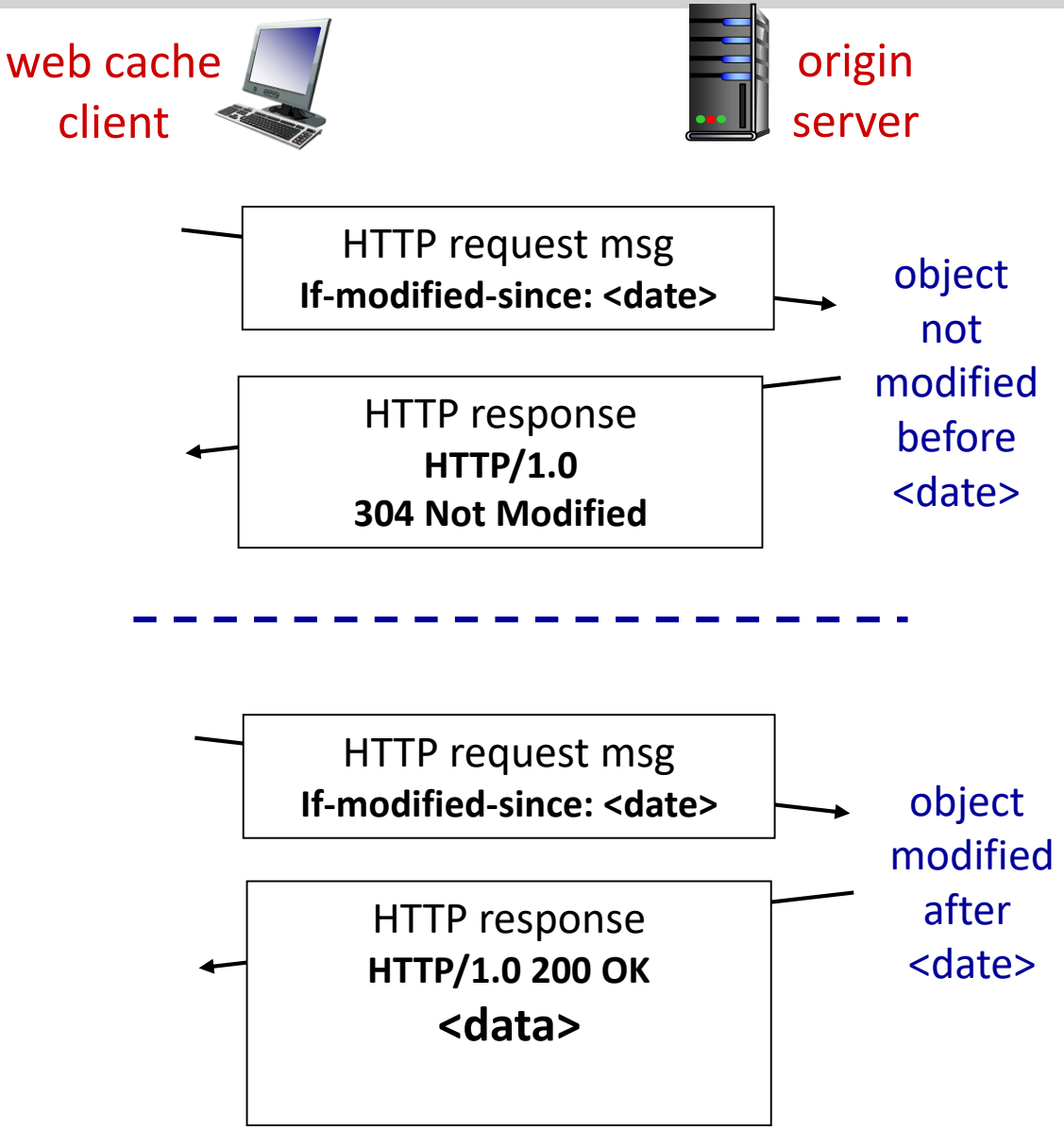
  - *server*: response contains no object if cached copy is up-to-date:

    HTTP/1.0 304 Not Modified

Cache-Control: max-age=<seconds>

Cache-Control: no-cache

Cache-Control: no-store



HTTP/2

Key goal: decreased delay in multi-object HTTP requests

HTTP/2: increased flexibility at server in sending objects to client:

- methods, status codes, most header fields unchanged from HTTP 1.1
- 요청 순위 우선화: transmission order of requested objects based on client-specified object priority (not necessarily FCFS)
- 서버 푸시: push unrequested objects to client
- 효율적인 헤더 전송: 중복 헤더 필드 제거 및 허프만 알고리즘으로 헤더 압축
- 프레이밍 (framing): divide objects into frames, schedule frames to mitigate HOL blocking

전자메일 (E-mail)

사용자 에이전트

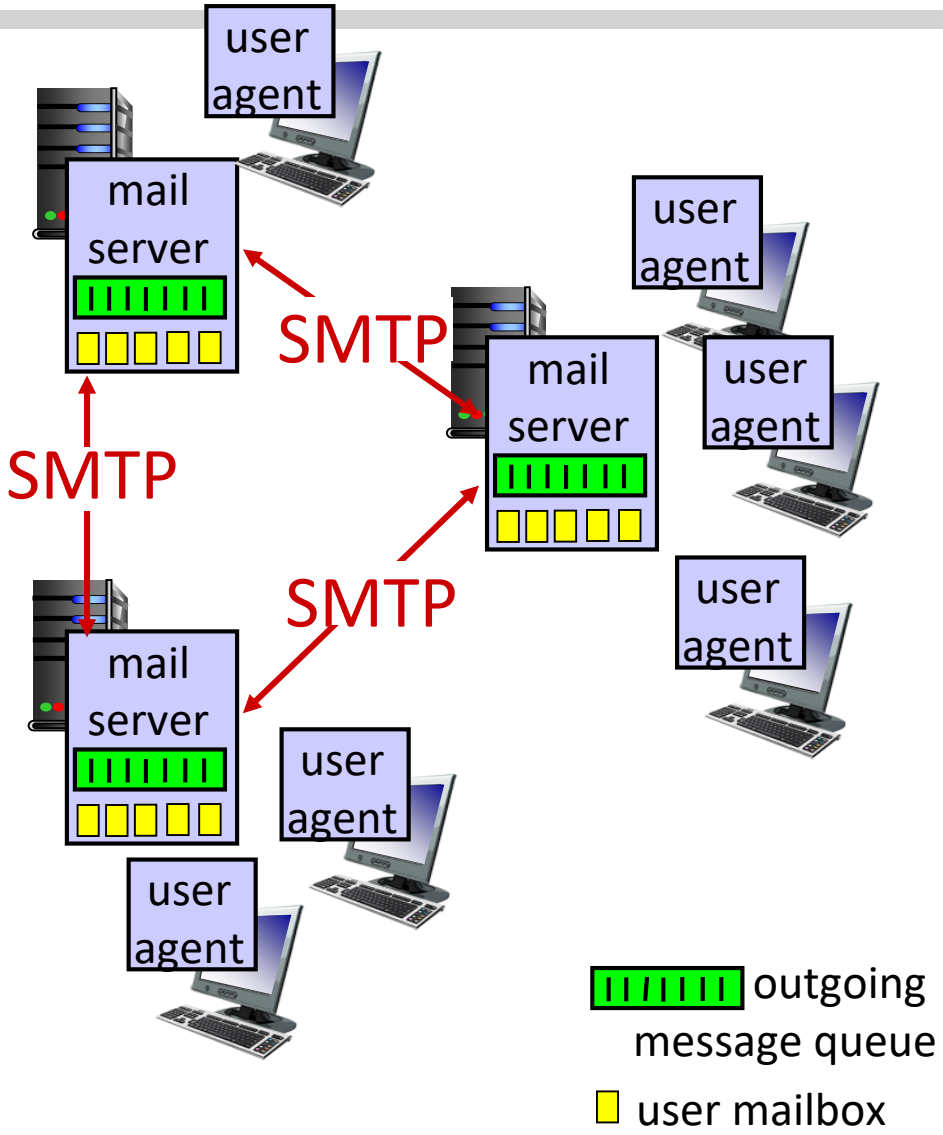
- 메일 메시지 작성, 편집, 읽기
- 발신(outgoing), 수신(incoming) 메시지는 서버에 저장됨

메일 서버 (mail servers):

- mailbox contains incoming messages for user
- message queue of outgoing (to be sent) mail messages

SMTP between mail servers to send email messages

- client: sending mail server
- server: receiving mail server

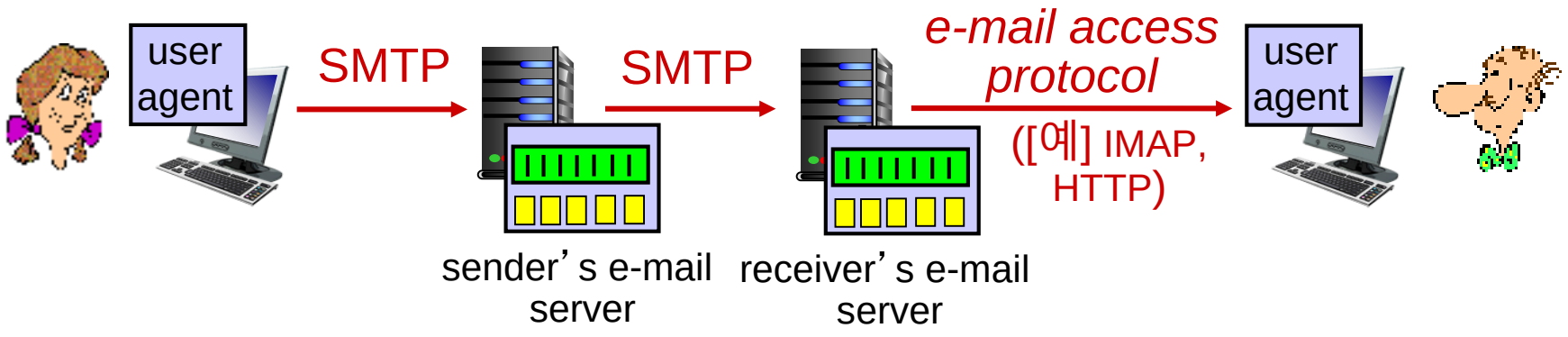


SMTP: observations

comparison with HTTP:

- both have ASCII command/response interaction, status codes
- HTTP: client pull (on-demand, 데이터를 서버가 전송)
- SMTP: client push (데이터를 클라이언트가 전송)
- HTTP: 각 객체는 각 응답 메시지에서 캡슐화
- SMTP: 하나의 메시지에 여러 객체가 포함
- SMTP는 지속 연결 사용
- SMTP의 메시지는 7비트 ASCII 문자로만 구성
  - 멀티미디어 데이터를 ASCII 문자로 변환 후 전송 → 전송 후 원래 메시지로 변환
- SMTP 서버는 ‘CRLF.CRLF’를 통해 메시지 끝을 확인

Retrieving email: 메일 접속 프로토콜 (mail access protocols)



- SMTP: delivery/storage of e-mail messages to receiver's server
- mail access protocol: retrieval from server: POP3, IMAP, HTTP

DNS: Domain Name System

IP 주소:

- 일반적인 IP 주소 (IPv4)의 길이는 32비트이며, 0~255 사이의 4개 숫자로 표시
- 고정된 길이의 계층적 구조

Internet hosts, routers:

- IP 주소를 데이터그램 주소 지정에 사용

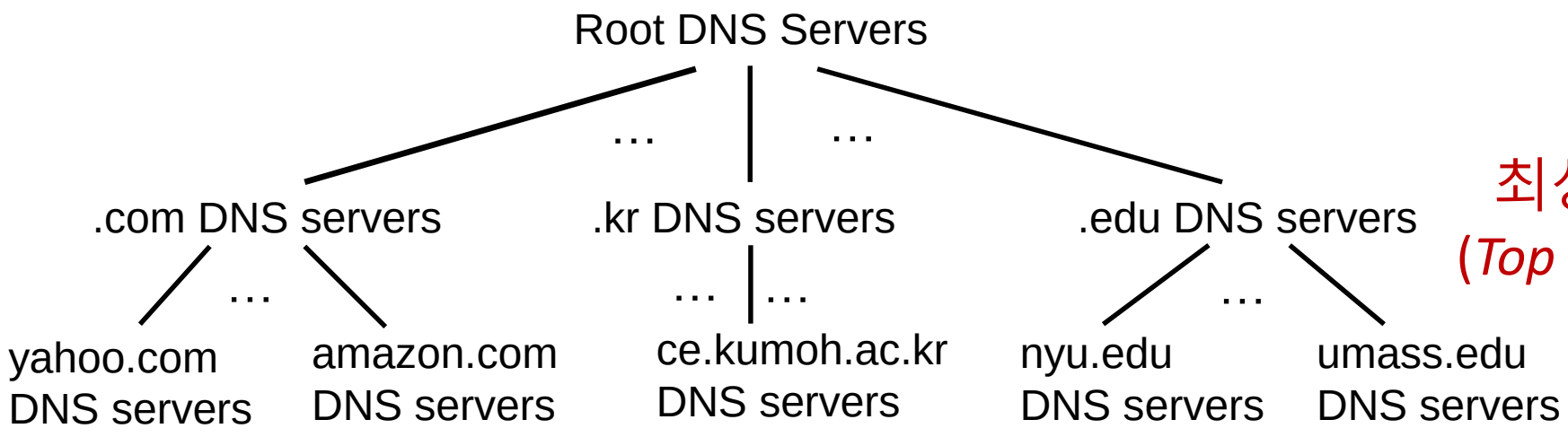
호스트 이름 (hostname):

- 사람이 사용, [예] ce.kumoh.ac.kr

Domain Name System (DNS):

- DNS 서버들의 계층 구조로 구현된 분산 데이터베이스
- 호스트가 분산 데이터베이스로 질의(query)을 허용하는 애플리케이션 계층 프로토콜
- 애플리케이션 계층 프로토콜로 구현된 핵심 인터넷 기능
  - UDP상에서 수행되고 포트번호는 53번
- DNS 서비스:
  - 호스트 이름을 IP 주소로 변환
  - 호스트와 메일 서버 별칭
  - 부하 분산

DNS: 분산 계층 데이터베이스



루트
(Root)

최상위 레벨 도메인
(*Top Level Domain*, TLD)

책임
(*authoritative*)

로컬 DNS 서버가 www.amazon.com의 IP 주소를 요구 (처음 접근):

- 루트 DNS 서버에 질의하여 .com DNS 서버 (TLD 서버의 IP 주소)를 찾음
- .com DNS 서버를 질의하여 amazon.com DNS 서버 (책임 서버의 IP 주소)를 찾음
- amazon.com DNS 서버에 질의하여 www.amazon.com의 IP 주소를 획득

로컬 DNS 서버 및 DNS 질의 방법

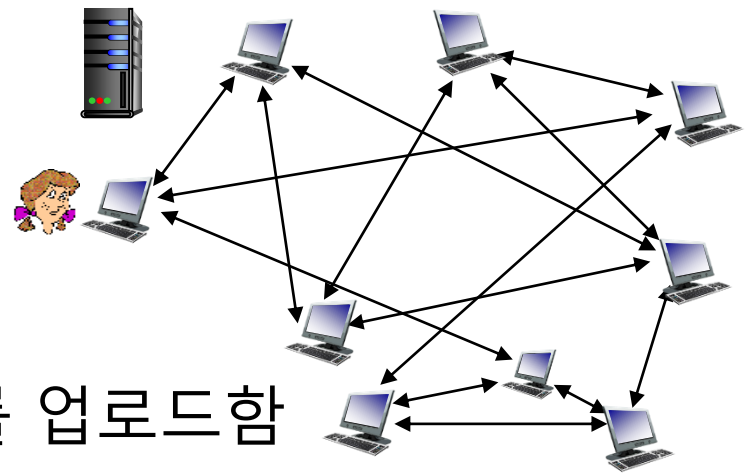
- 호스트의 DNS 질의는 기본적으로 로컬 DNS 서버로 전달
- 로컬 DNS 서버는 프록시 역할을 수행하여, 질의를 DNS 서버 계층으로 전달
  - 만약, 호스트가 질의한 호스트이름이 로컬 DNS 서버에 (호스트이름, IP주소) 짝으로 캐싱되었다면, 해당 IP 주소를 호스트에게 즉시 반환 (최신 IP 주소가 아닐 수 있음)
  - 각 ISP에는 로컬 DNS 서버가 존재
- 로컬 DNS 서버는 DNS 서버들의 계층구조에 엄격하게 속하지 않지만, DNS 구조의 중심에 있음
- 반복적 질의 (Iterated query) vs. 재귀적 질의 (Recursive query)

DNS 캐싱

- DNS 서버는 호스트 이름과 IP주소의 짝을 캐싱하여, 같은 질의에 대한 응답으로 캐시된 IP 주소를 즉시 전달
  - 캐싱으로 응답 시간과 인터넷 상의 DNS 메시지 수를 감소
  - cache entries timeout (disappear) after some time (time to live, TTL)
  - 일반적으로 로컬 DNS 서버에 TLD DNS 서버의 IP 주소들이 캐싱되어 있음
- cached entries may be *out-of-date*
  - if named host changes IP address, may not be known Internet-wide until all TTLs expire!
  - *best-effort hostname-to-IP address translation!*

P2P 파일 분배: BitTorrent

- 피어가 처음으로 토렌트 가입:
  - 트래커에 등록하여 피어 목록을 가져오고, 임의의 피어 부분 집합 (이웃 피어들)과 'TCP' 연결
  - 청크가 없지만 시간이 지남에 따라 다른 피어로부터 축적



- 피어는 다운로드하는 동안 다른 피어들에게 청크를 업로드함
- 청크를 교환하는 피어들은 변경될 수 있음: peers may come and go
- 피어가 전체 파일을 얻으면, 토렌트를 (이기적으로) 떠나거나 혹은 (이타적으로) 토렌트에 남아 청크를 계속 업로드할 수 있음
  - 무료 승차족 (free-rider): 피어는 일반적으로 다른 피어에게 파일을 보내지 않고 파일을 받기를 원함

BitTorrent: requesting, sending file chunks

청크 요청 방식 (Requesting chunks):

- 임의의 시간에 각 피어는 서로 다른 파일 청크들을 가짐
- 주기적으로 Alice는 각 피어에게 가지고 있는 청크 목록을 요청
- Alice는 피어들에게 갖고 있지 않는 청크를 요청
  - 일반 P2P 다운로드: 가장 희귀한 청크부터 요청 (희귀 청크 우선 전략, rarest-first)
  - 스트리밍 기반 P2P: 곧 재생될 화면에 필요한 청크부터 요청 (재생 우선 전략)

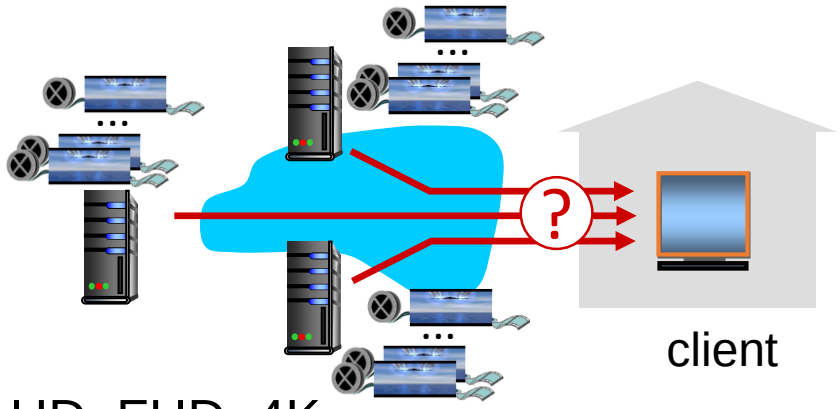
청크 전송 방식 (Sending chunks): tit-for-tat

- Alice는 현재 가장 빠른 속도로 청크를 보내는 4개의 피어에게 자신의 청크를 보냄으로써 보답 ('활성화')
  - Alice는 다른 피어에게 청크를 보내지 않음 ('비활성화')
  - 10초마다 속도를 재평가하여 상위 4개 피어 집합을 수정
- 30초마다 무작위로 다른 피어를 추가로 선택하고 청크를 먼저 전송 ('낙관적 활성화')
  - 새로 선택된 피어가 상위 4개의 피어 중 하나가 될 수 있음

Streaming multimedia: DASH

비디오 품질이 좋을수록 더 높은 대역폭이 필요

Dynamic, Adaptive Streaming over HTTP:



- Server:
  - 비디오 파일을 여러 비디오 조각 (chunk)으로 분할
  - 각 조각은 여러 품질 버전으로 인코딩 (encoding)됨: [예] SD, HD, FHD, 4K
  - 여러 품질 버전의 비디오 파일들은 여러 서버에서 저장
  - 매니페스트 (*manifest*) 파일: 여러 품질 버전의 비디오 조각 URL 목록 정보를 제공
- Client:
  - 주기적으로 서버-클라이언트 간 대역폭을 측정
  - 매니페스트 파일을 참조하여, 한번에 한 비디오 조각을 요청
    - 현재 대역폭에서 지속 가능한 최대 품질 버전의 비디오 조각을 선택
    - 가용한 대역폭 상황에 따라 시점마다 다른 서버와 다른 품질 버전을 요청할 수 있음

Streaming video = encoding + DASH + playout buffering

Content distribution networks (CDNs)

수십만 명의 동시 사용자에게 콘텐츠 (수천만 개의 비디오 중 하나를 선택)를 스트리밍하는 방법은?

- 지리적으로 분산된 여러 클러스터에서 여러 비디오 사본을 저장/제공 (CDN, 콘텐츠 분배 네트워크 또는 콘텐츠 제공 네트워크)
  - 콘텐츠 제공자가 소유한 private CDN과 제 3자가 운영하는 third-party CDN으로 구분
  - *enter deep*: CDN 서버를 다수의 접속 네트워크에 설치
    - 사용자와 최대한 가깝게 클러스터를 설치하여 링크와 라우터의 수를 줄이고, 지연시간 및 처리율을 개선 (개별 클러스터의 규모는 작음)
    - used by Akamai: 360,000 servers deployed in 135 countries (over 4,200 locations)
  - *bring home*: CDN 서버를 소수(수십개)의 IXP에 설치
    - 클러스터 유지 및 관리 비용 감소 (개별 클러스터의 규모가 큼)
    - used by Limelight

Akamai

Limelight NETWORKS

Socket Programming

TCP vs UDP 코드 차이

Python TCPServer

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
print ('The server is ready to receive')
while True:
    connectionSocket, addr = serverSocket.accept()
    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.encode())
    connectionSocket.close()
```

Python UDPServer

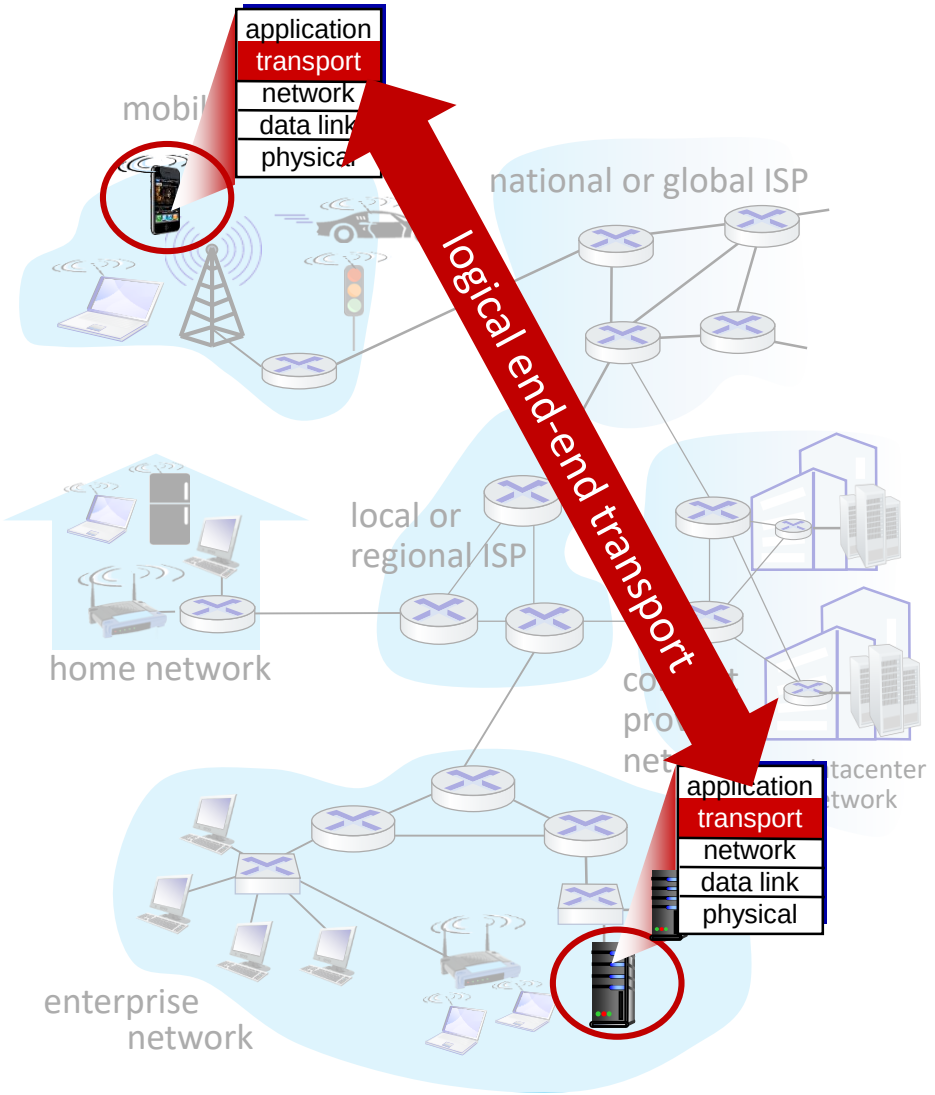
```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print ('The server is ready to receive')
while True:
    message, clientAddress = serverSocket.recvfrom(1024)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(),
                        clientAddress)
```

3. Transport Layer

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality

Transport services and protocols

- 다른 호스트에서 동작하는 응용 프로세스 간의 논리적 통신을 제공
  - 응용 프로세스의 관점에서 호스트들이 직접 연결된 것처럼 보임
- 트랜스포트 프로토콜은 종단 시스템 (호스트)에서 동작:
  - 송신자 (sender): 응용 프로그램의 메시지를 세그먼트 (segment)로 나누고 네트워크 계층으로 전달
  - 수신자 (receiver): 세그먼트들을 메시지로 재조립하여 응용 계층으로 전달



Transport Layer Actions

Sender:

- is passed an application-layer message
- determines segment header fields values
- creates segment
- passes segment to IP

Receiver:

- receives segment from IP
- checks header values
- extracts application-layer message
- demultiplexes message up to application via socket

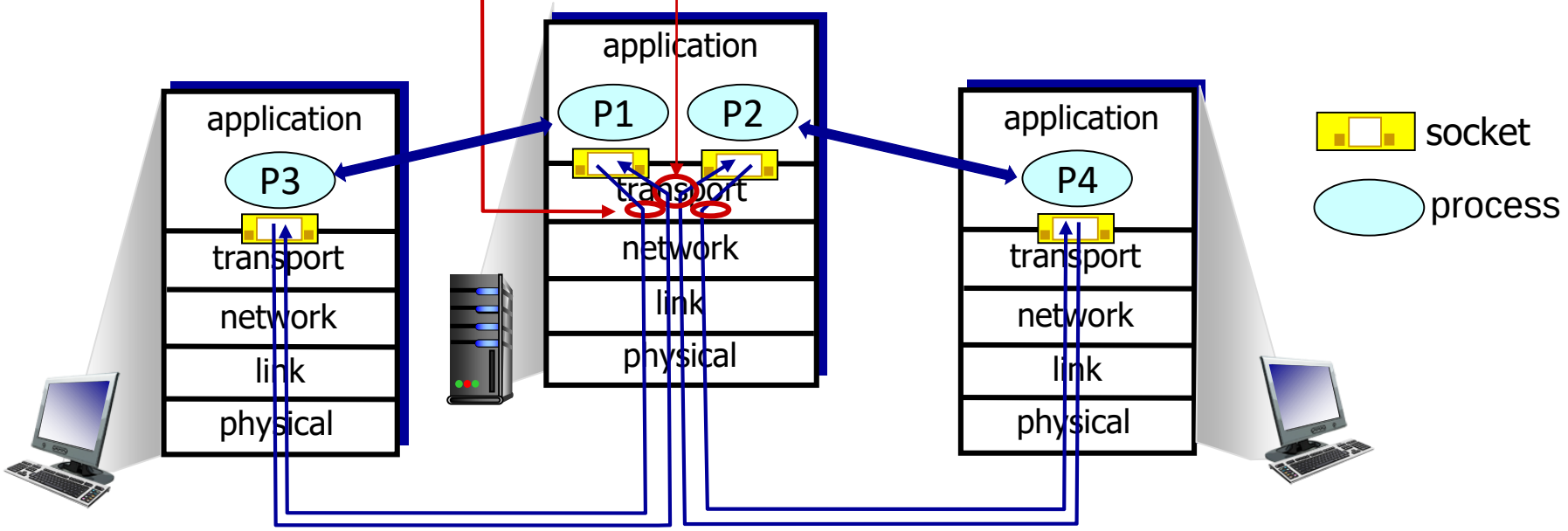
Multiplexing/demultiplexing

다중화 (multiplexing) at sender:

여러 소켓으로부터 메시지를 수집하고, 세그먼트 단위로 메시지를 분리하고, 각 메세지를 캡슐화해서 네트워크 계층으로 전달 (추후 역다중화에 사용)

역다중화 (demultiplexing) at receiver:

헤더 정보를 확인하여 수신된 세그먼트의 메세지를 올바른 소켓으로 전달



Multiplexing/demultiplexing

다중화, 역다중화: 세그먼트와 데이터그램 헤더 필드 값을 활용

- 호스트는 IP 주소와 포트 번호를 활용하여 세그먼트를 적절한 소켓으로 전달
- UDP: 목적지 포트 번호만 사용하여 역다중화
- TCP: 4개 정보를 사용한 역다중화: 출발지 및 목적지의 IP 주소와 포트 번호
- Multiplexing/demultiplexing happen at all layers

UDP: User Datagram Protocol

- 최소 기능 인터넷 트랜스포트 프로토콜
- 'best effort' service
  - lost, delivered out-of-order to app
- *Connectionless* (비연결형):
  - no handshaking between UDP sender and receiver
  - 각 UDP 세그먼트는 서로 독립적으로 처리됨
- UDP use:
  - streaming multimedia apps, DNS, HTTP/3
  - if reliable transfer needed over UDP ([예], HTTP/3):
  - add needed reliability at application layer
  - add congestion control at application layer

Why is there a UDP?

- 연결 설정 없음 (RTT 지연 없음)
- 송신자, 수신자의 연결 상태 없음 (각종 파라미터 등)
- 작은 헤더 사이즈 (8 Bytes)
- 혼잡 제어 없음
  - UDP는 응용 프로그램이 원하는 만큼 빠르게 전송 가능
  - 네트워크 혼잡이 발생해도 동작할 수 있음
    - TCP 연결의 성능을 크게 저해할 수 있음

Internet checksum: an example

example: add two 16-bit integers

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound (윤회식 자리올림)	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

참고: 16비트씩 더할 때는 발생하는 가장 왼쪽 비트의 캐리 (carry)는 윤회식 자리올림으로 결과 값의 가장 오른쪽 비트에서 더함

RDT 2.0: 비트 오류가 발생하는 채널

- *RDT 2.0*: 하위 채널에서 전송 중 패킷 안의 비트 오류가 발생할 수 있음
  - 비트 오류를 탐지하기 위한 체크섬 (checksum)을 추가
- 하지만, 송신자는 패킷을 성공적으로 전송했는지 알 수 없음
- *acknowledgements* (ACKs, 긍정 확인응답): 수신자는 송신자에게 패킷을 정상적으로 수신되었음을 명시적으로 알림
- *negative acknowledgements* (NAKs, 부정 확인응답): 수신자는 송신자에게 수신한 패킷에 오류가 있음을 명시적으로 알림
  - NAK을 수신한 송신자는 패킷을 재전송

Stop-and-wait (전송 후 대기)

송신자가 하나의 패킷을 전송한 후 수신자의 응답을 기다림

RDT 2.1: 손상된 ACK/NAK 처리

만약 ACK/NAK에서 비트 오류가 발생한다면?

- ACK/NAK 패킷들의 오류를 탐지를 위한 체크섬 추가
- 수신자의 피드백을 받더라도, 송신자는 패킷 전송 성공 여부를 알 수 없음!
- 단순히 재전송할 수 없음: 패킷이 중복 전송될 수 있음

중복 처리:

- 송신자는 ACK/NAK가 손상된 경우 현재 패킷을 재전송
- 송신자는 각 패킷에 순서 번호 (sequence number)를 추가
  - 데이터를 순서대로 응용 계층에 전달 가능
  - 데이터 중복 방지
- 수신자는 순서 번호가 같은 중복 패킷을 버림 (응용 계층에 전달하지 않음)
  - 수신자는 중복 패킷을 버리더라도, 송신자에게 ACK는 보냄

RDT 3.0: 비트 오류와 패킷 손실이 있는 채널

새로운 채널 가정: 하위 채널에서 패킷 손실 (packet loss)이 발생할 수 있음 (data, ACKs)

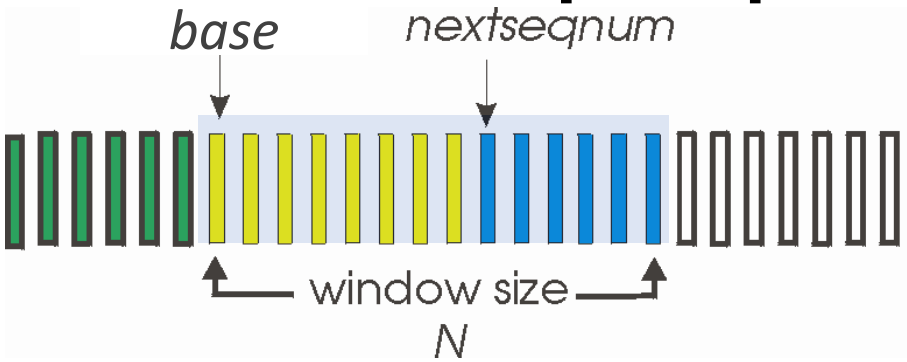
- 체크섬, 순서 번호, ACK, 재전송으로 충분하지는 않음
- 패킷 손실을 어떻게 탐지할 것인가?

Approach: 송신자는 '합리적인' 시간 동안 ACK를 기다림

- 합리적인 시간 내에 수신되는 ACK가 없으면 재전송
- 패킷(또는 ACK)이 단순히 지연된 경우 (패킷 손실이 아닐 경우):
  - 중복된 패킷이 재전송되지만, '순서 번호'로 처리 가능!
  - 수신자는 성공적으로 수신된 패킷의 순서 번호를 기억함
- 카운트다운 타이머 (countdown timer)를 사용하여 '합리적인' 시간을 대기 후 송신자에게 타임아웃 (인터럽트) 발생

Go-Back-N (GBN, N부터 반복): sender

- 송신자: 연속적으로 전송했지만, 아직 확인 응답이 안된 패킷들을 버퍼
  - 확인 응답없이 연속적으로 전송 가능한 버퍼 (window)의 크기 = *N*
  - 패킷 헤더의 *k* bit의 순서 번호 [0, 2^k- 1]가 포함



[0, base-1]

이미 응답됨

전송 후, 확인 응답 안 됨

[base, nextseqnum-1]

[nextseqnum, base+N-1]

전송 가능하지만 아직 전송 안됨

전송 불가

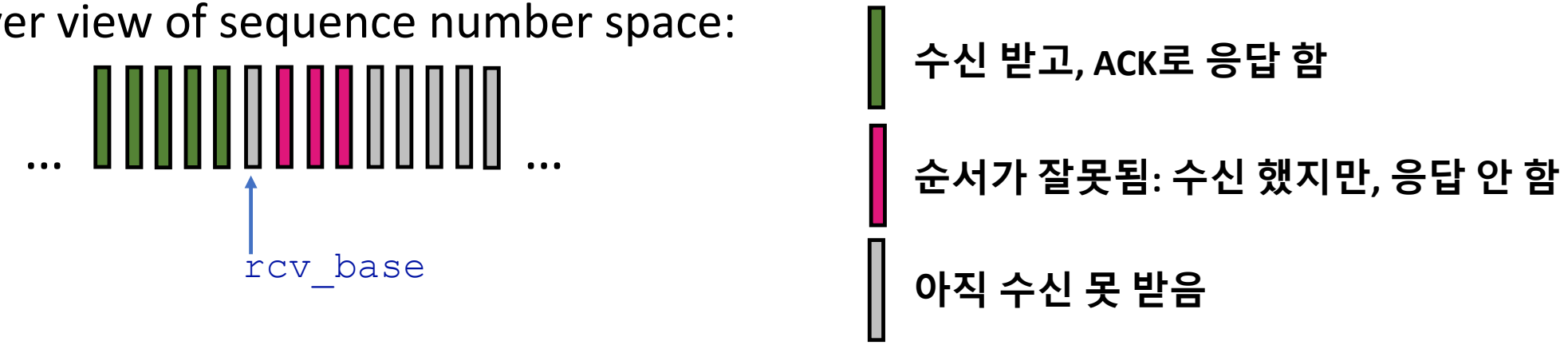
[base+N~]

  - base: 전송 되었지만, 확인 응답 안된 가장 오래된 패킷의 순서 번호 (timeout 발생 기준)
  - nextseqnum: 전송되지 않는 가장 작은 패킷의 순서 번호로, 전송될 다음 패킷의 순서 번호
- cumulative ACK (누적 확인 응답): 순서 번호 *n*의 확인 응답을 받으면, 순서 번호 *n*번까지의 모든 패킷에 대한 확인 응답
  - *n*번 확인 응답 수신: base가 n+1에서 시작하도록 window을 앞으로 이동 (sliding)
- timeout 발생: 전송 후, 확인 응답 안된 모든 패킷 [base, nextseqnum -1]을 재전송

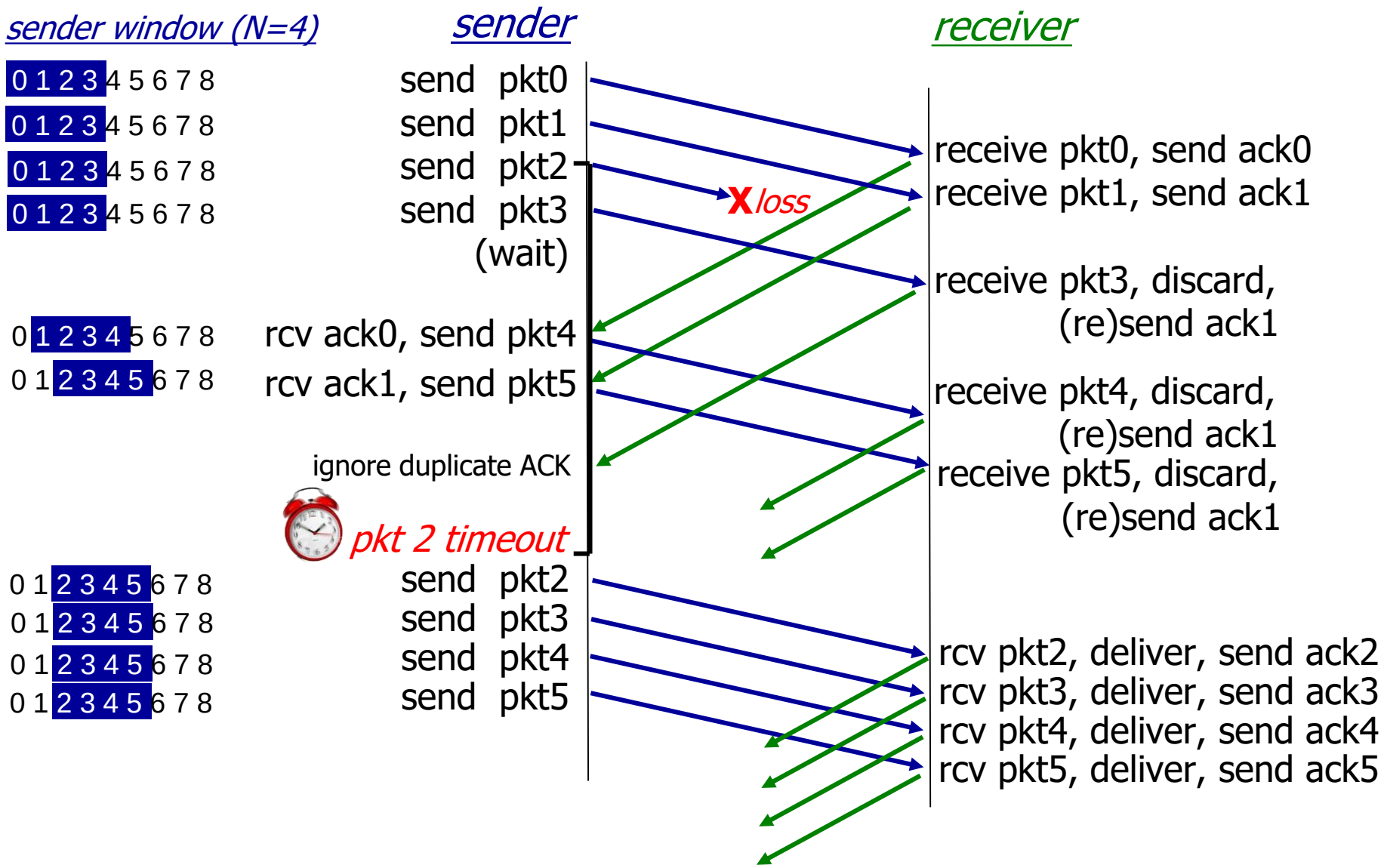
Go-Back-N: receiver

- 수신자: 지금까지 순서에 맞게 수신된 패킷의 가장 높은 순서 번호로 ACK를 보냄
  - 중복 확인 응답이 생성될 수 있음
  - 지금까지 순서에 맞게 수신된 패킷의 다음 순서 번호 (rcv_base)만 기억하면 됨
- 순서가 잘못된 패킷을 수신하면:
  - 폐기하거나 버퍼할 수 있음: 개발자가 구현할 때 결정
  - 순서대로 받은 가장 높은 번호를 가진 패킷에 대한 ACK를 재전송

Receiver view of sequence number space:



Go-Back-N in action

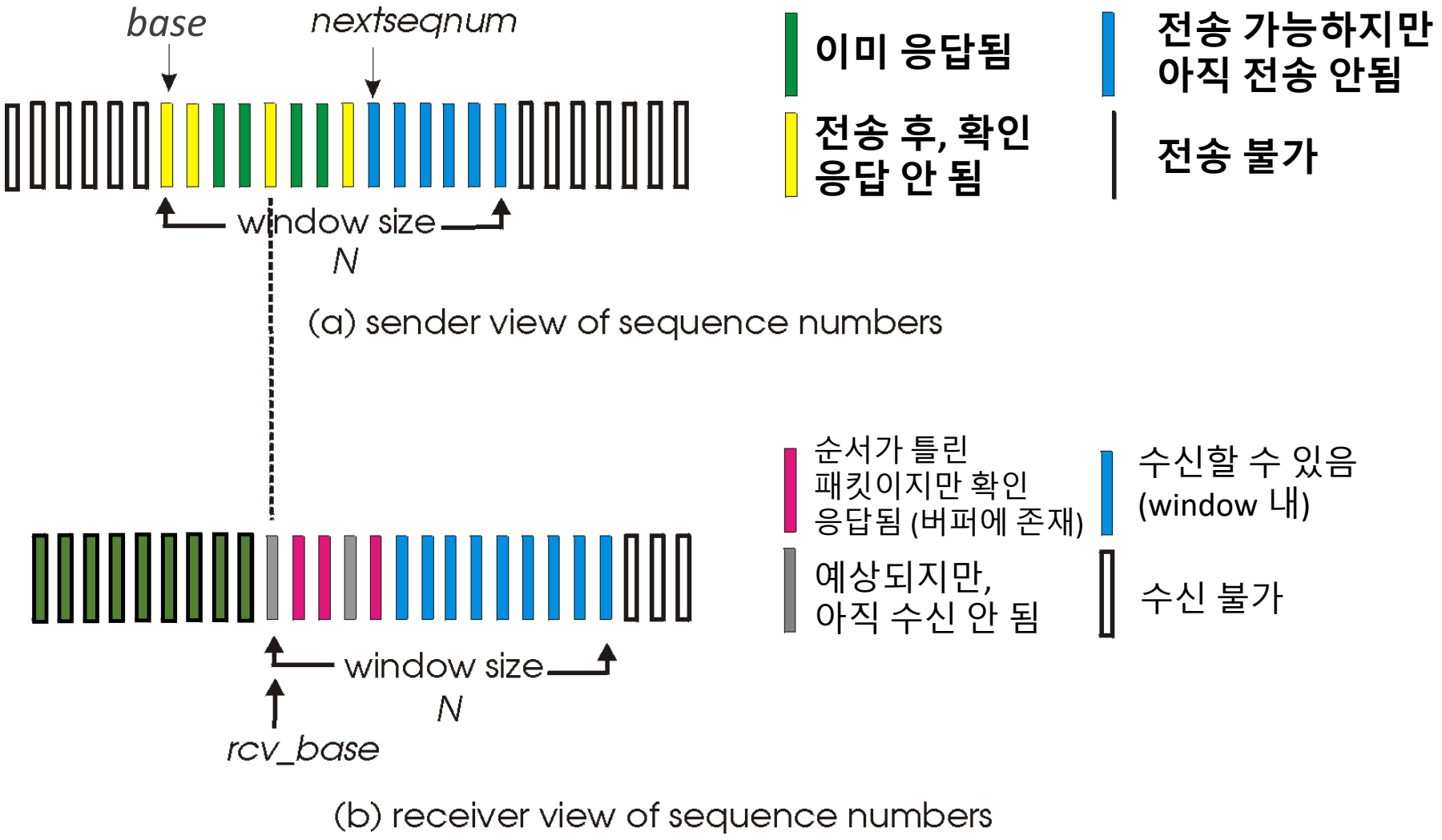


Selective Repeat (SR, 선택적 반복)

- 송신자: 확인 응답이 안된 패킷에 대해 개별적으로 timeout과 재전송을 수행
  - 송신자는 확인 응답되지 않은 각 패킷에 대해 개별 타이머를 유지
  - timeout이 발생하면 해당 패킷만 재전송
- 송신자 window: 연속된 *N*개의 순서 번호를 가진 패킷들로 구성
  - 확인 응답 안된 패킷의 수를 제한

- 수신자: 올바르게 수신된 모든 패킷을 개별적으로 확인 응답함
  - 응용 계층에게 순서대로 전달을 위해, 필요에 따라 패킷을 버퍼함

Selective repeat: sender, receiver windows



Selective repeat: sender and receiver

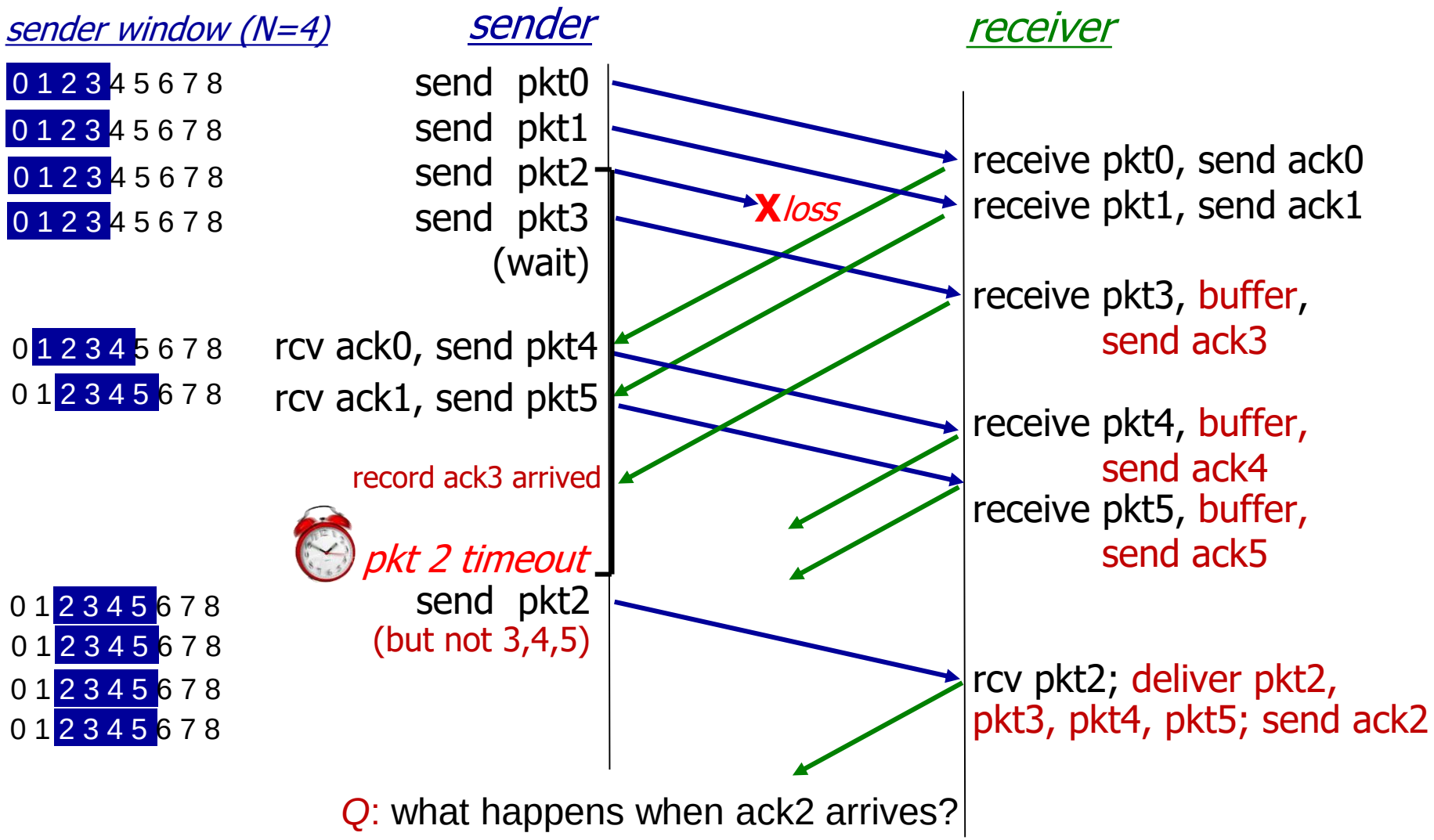
sender

- 애플리케이션으로부터 데이터 수신:
  - 만약 순서 번호가 window 내에 있으면 패킷을 전송
- timeout(*n*):
  - *n*번 패킷만 다시 전송하고, timer를 재시작
- ACK(*n*) in [base, base+N-1]:
  - *n*번 확인 응답 받았다고 표기
  - 확인 응답된 *n*번 패킷이 base인 경우, window의 base를 확인 응답되지 않은 가장 낮은 순서 번호의 패킷이 되도록 이동

receiver

- packet *n* in [rcvbase, rcvbase+N-1]:
  - *n*번 ACK 전송
  - 순서가 맞지 않음 (out-of-order): buffer
  - 순서가 맞음 (in-order): 응용 계층에 전달 (또한 버퍼링된 패킷을 순서대로 전달), 아직 수신되지 않은 다음 패킷으로 rcvbase를 이동
- packet *n* in [rcvbase-N, rcvbase-1]:
  - *n*번 ACK 전송
- otherwise: 무시

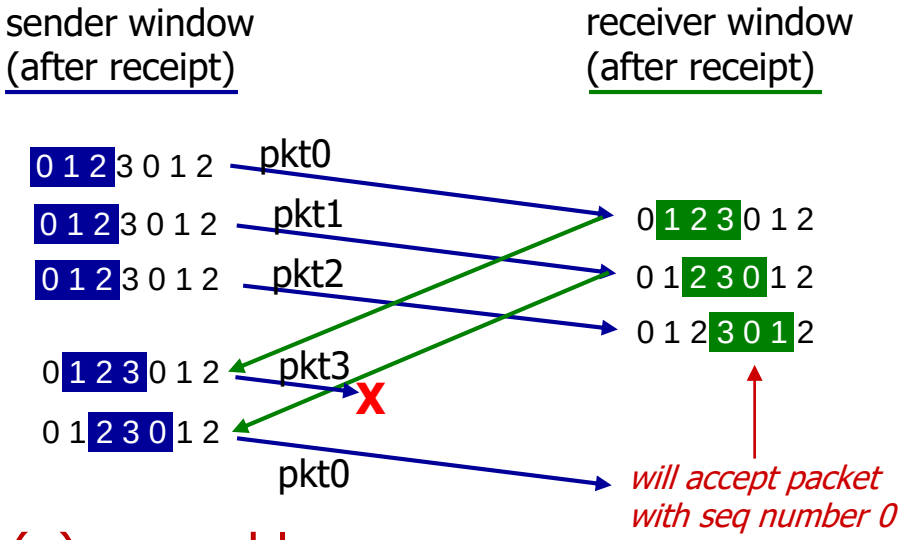
Selective repeat in action



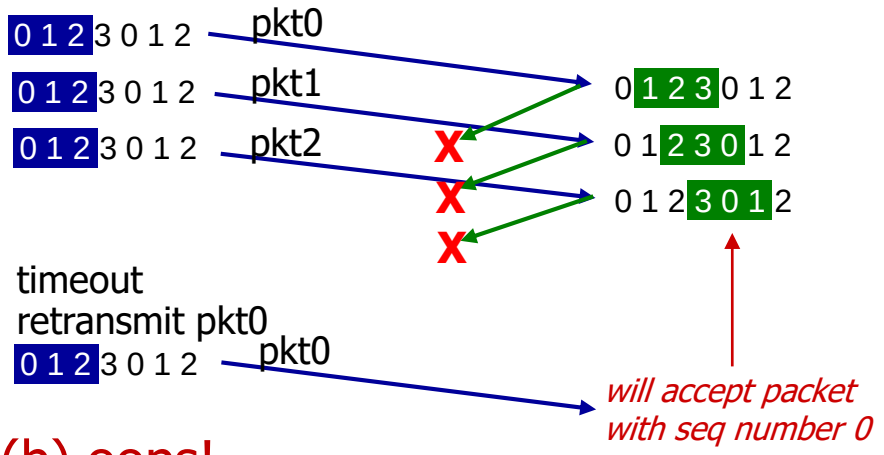
Selective repeat: a dilemma!

[예] Selective repeat와 순서번호

- 순서 번호: 0, 1, 2, 3 (base 4 counting)
- window size = 3



(a) no problem

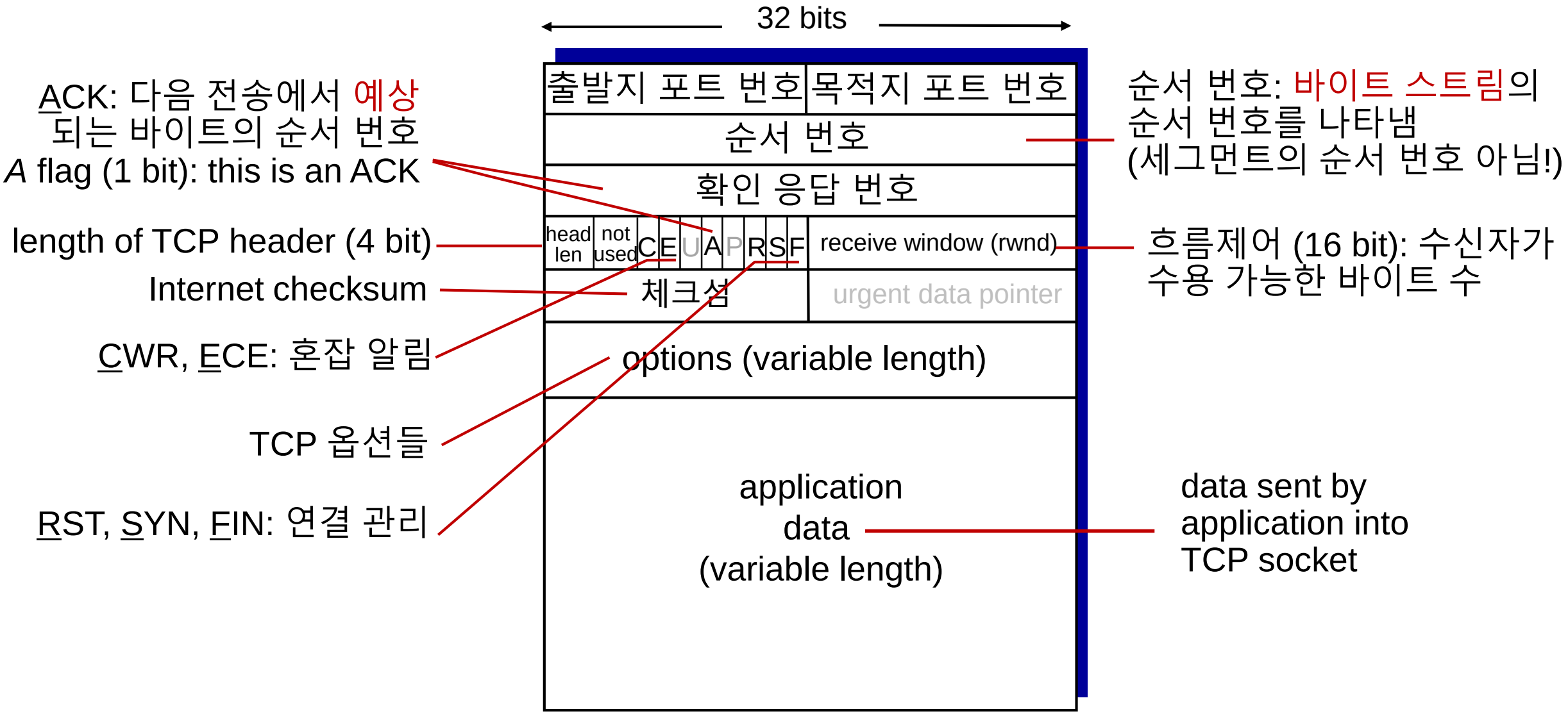


(b) oops!

TCP 개요 (RFCs: 793,1122, 2018, 5681, 7323)

- 점대점 (point-to-point):
  - one sender, one receiver (unicast)
- reliable, logical in-order byte stream:
  - 메시지 '경계'가 없음
- 전이중 서비스 (full duplex service):
  - 동일한 연결에서 양방향으로 데이터가 전송
  - maximum segment size (MSS): 최대 세그먼트 크기 (응용 계층 데이터의 최대 크기)
- 누적 확인응답 (cumulative ACKs)
- pipelining:
  - TCP 혼잡 제어와 흐름 제어를 기반으로 윈도우 크기가 설정됨
- connection-oriented:
  - 핸드셰이킹 (제어 메시지 교환)은 데이터 교환 전에 송신자, 수신자 상태 초기화
- flow controlled:
  - 송신자는 수신자의 윈도우 상태를 확인하면서 데이터를 전송

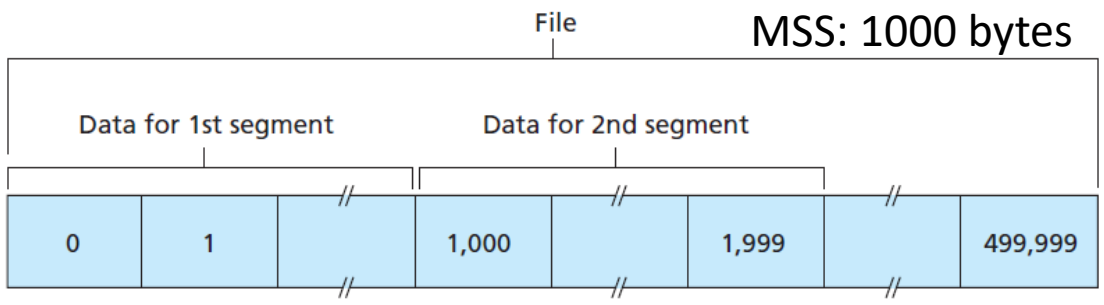
TCP segment structure



TCP sequence numbers, ACKs

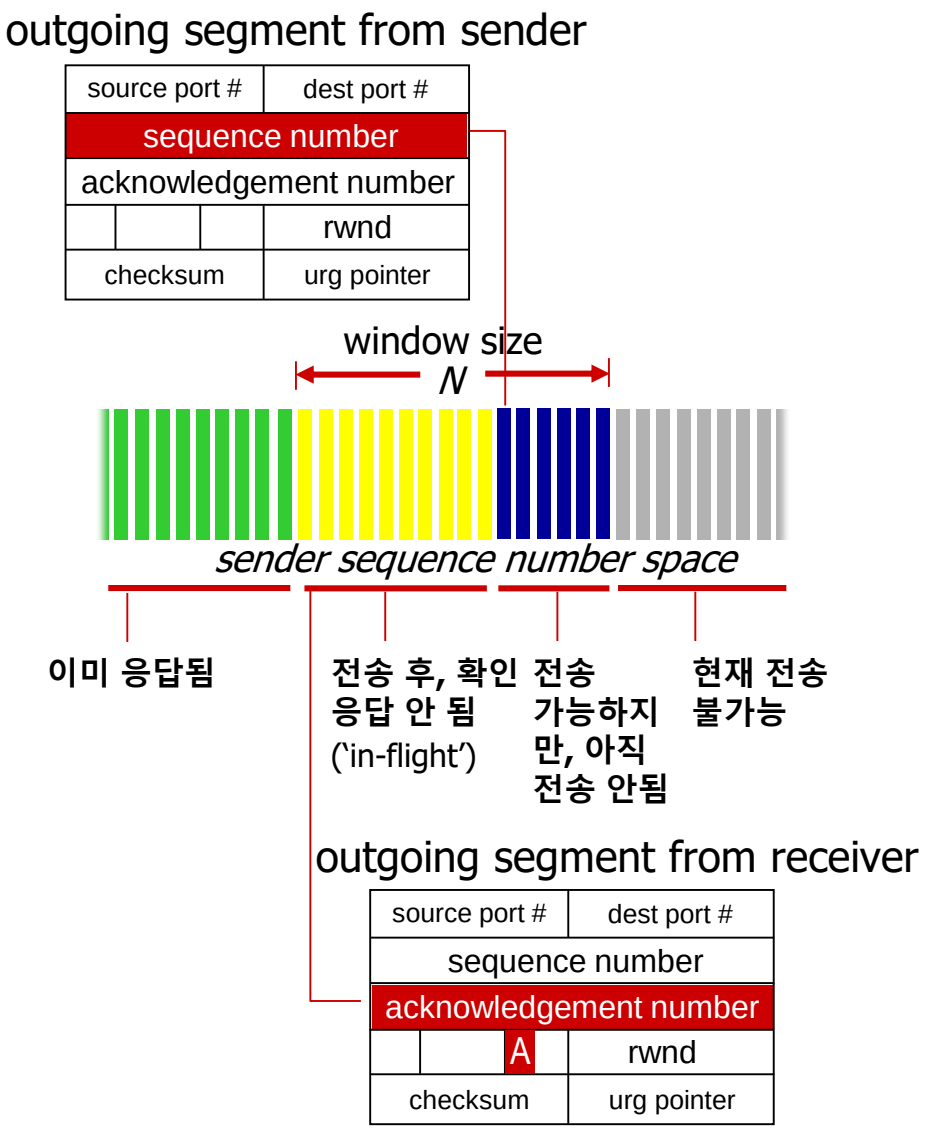
Sequence numbers:

- 세그먼트 데이터에서 바이트 스트림의 첫 번째 바이트 '숫자'



Acknowledgements:

- 다음 전송에서 수신될 것으로 예상되는 바이트의 순서 번호
- cumulative ACK



TCP round trip time, timeout

Q: TCP 타임 아웃 값을 어떻게 설정?

- RTT 보다 길어야 되지만, 문제는 RTT가 계속 변함!
  - too short: 조기 타임아웃으로 불필요한 재전송 발생
  - too long: 세그먼트 손실에 대한 느린 반응

Q: 그럼 RTT를 어떻게 추정?

- *SampleRTT*:세그먼트 전송부터 ACK 수신까지 측정된 시간 (Timestamp option)
  - RTT 측정에서 재전송 세그먼트는 배제
- *SampleRTT*는 계속 변하므로 '평준화'된 추정 RTT가 필요함
  - 현재 *SampleRTT*뿐만 아니라 여러 최근 측정값의 평균을 활용

$$\mathbf{DevRTT = (1 - \beta) \times DevRTT + \beta \times |SampleRTT - EstimatedRTT|}$$

$$\mathbf{EstimatedRTT = (1 - \alpha) \times EstimatedRTT + \alpha \times SampleRTT}$$

$$\mathbf{TimeoutInterval = EstimatedRTT + 4 \times DevRTT}$$

TCP Sender (simplified)

애플리케이션으로부터 데이터 수신

- 순서 번호가 포함된 세그먼트로 캡슐화
- 순서 번호는 세그먼트에서 첫 번째 데이터 바이트 스트림의 번호를 나타냄
- 타이머가 아직 실행되지 않으면 타이머를 시작
  - 확인 응답 안된 가장 오래된 세그먼트의 타이머
  - 만료 주기 (expiration interval): TimeOutInterval

타이머 타임아웃

- 타임아웃을 발생시킨 세그먼트 재전송
- 타이머 재시작
- expiration interval = 2 × TimeOutInterval
- 이후 정상적으로 세그먼트를 수신하면, TimeOutInterval 다시 계산

ACK 수신

- 이전에 확인 응답 되지 않은 세그먼트에 대한 ACK이면,
  - 해당 세그먼트를 확인 응답 표시 및 필요시 base 갱신
  - 확인 응답 안된 세그먼트가 존재한다면, 타이머를 재시작

TCP Receiver: ACK 생성 권고 [RFC 5681]

Event at receiver	*TCP receiver action*
기다리는 순서 번호를 가진 '순서가 맞는' 세그먼트의 도착. 기다리는 순서 번호까지의 모든 데이터는 이미 확인 응답됨	지연된 ACK: 또 다른 '순서가 맞는' 세그먼트의 도착을 위해 500 ms까지 기다림. 만약 다음 '순서에 맞는' 세그먼트가 이 기간에 도착하지 않으면, 그냥 ACK를 전송
기다리는 순서 번호를 가진 '순서가 맞는' 세그먼트의 도착. ACK 전송을 기다리는 다른 하나의 '순서에 맞는' 세그먼트가 있음	2개의 '순서가 맞는' 세그먼트 를 ACK 하기 위해, 하나의 누적된 ACK를 즉시 전송
기다리는 것보다 높은 순서 번호를 가진 '순서가 바뀐' 세그먼트의 도착. 격차 (gap)가 발견됨	순서 번호가 다음의 기다리는 바이트(즉, 격차의 최솟값)를 나타내는 중복 ACK를 즉시 전송
수신 데이터에서 격차를 부분적으로 또는 모두 채우는 세그먼트의 도착	해당 세그먼트가 격차의 최솟값에서 시작한다고 하면, 즉시 ACK를 전송

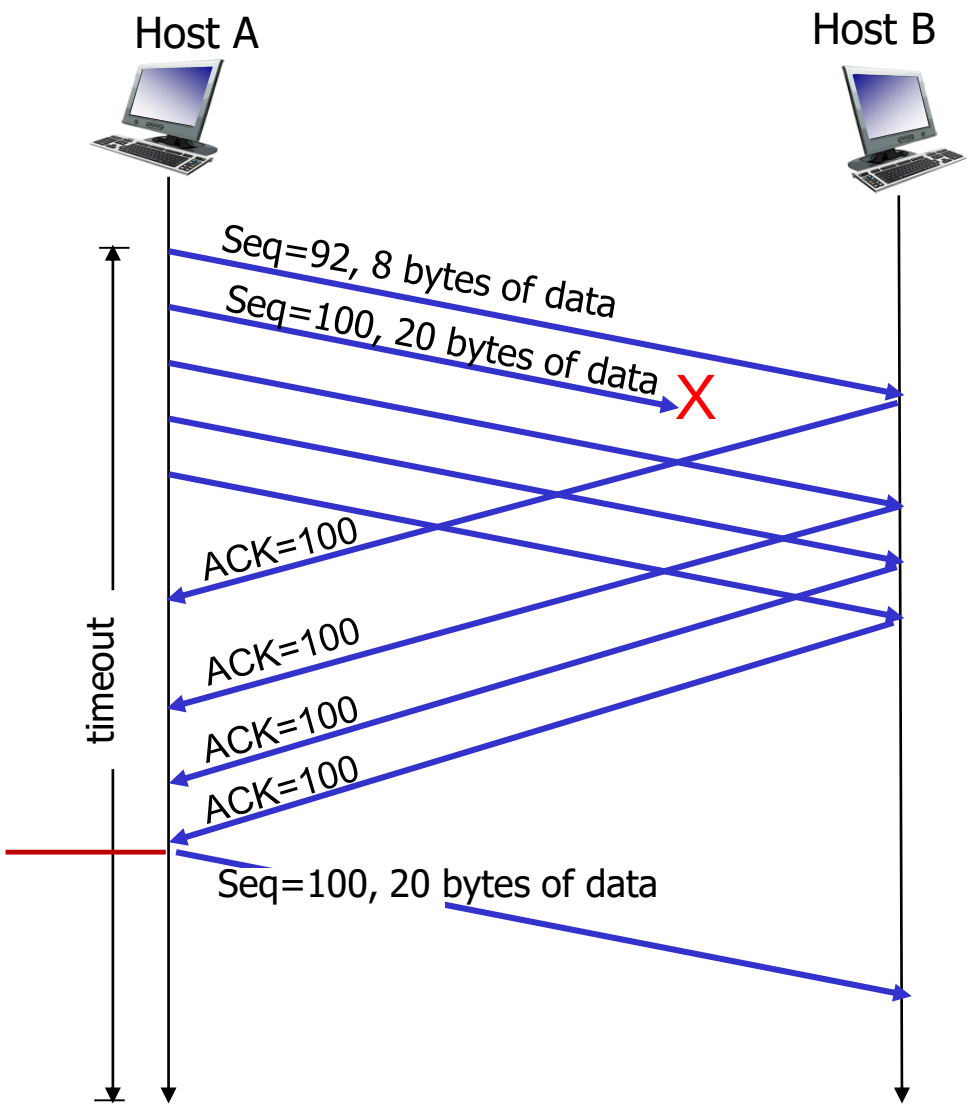
TCP 빠른 재전송 (fast retransmit)

TCP fast retransmit

송신자가 동일한 데이터에 대해 3개의 ACK를 추가로 수신한다면, ('3개의 중복 확인 응답'), 확인 응답 되지 않은 가장 작은 순서 번호의 세그먼트를 재전송

- 확인 응답되지 않은 세그먼트가 손실되었을 가능성이 높으므로 타임아웃까지 기다리지 않음

3개의 중복된 ACK 수신은, 누락된 세그먼트 이후에 3개의 세그먼트가 수신되었음을 의미함. 세그먼트가 손실되었을 가능성이 높기 때문에 재전송!

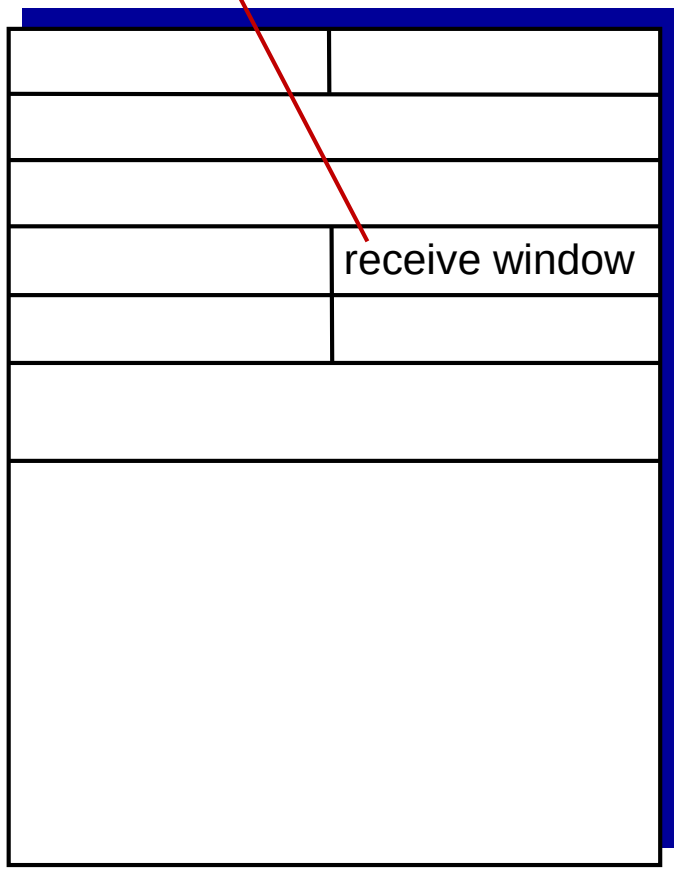


TCP flow control (흐름 제어)

- 수신자는 TCP 헤더의 *rwnd* 필드에 수용 가능한 버퍼의 크기를 '알림'
  - *RcvBuffer* size set via socket options (typical default is 4096 bytes)
  - 대부분의 운영체제는 *RcvBuffer*를 자동 조정
- 송신자는 수신자의 *rwnd*를 통해 확인응답 없이 연속적으로 전송할 수 있는 데이터의 양 (window size)을 제한
- 수신 버퍼에서 오버플로우가 발생하지 않는다는 것을 보장

Q: rwnd = 0이면?

flow control: 수신자가 현재 수용할 수 있는 바이트의 수



TCP segment format

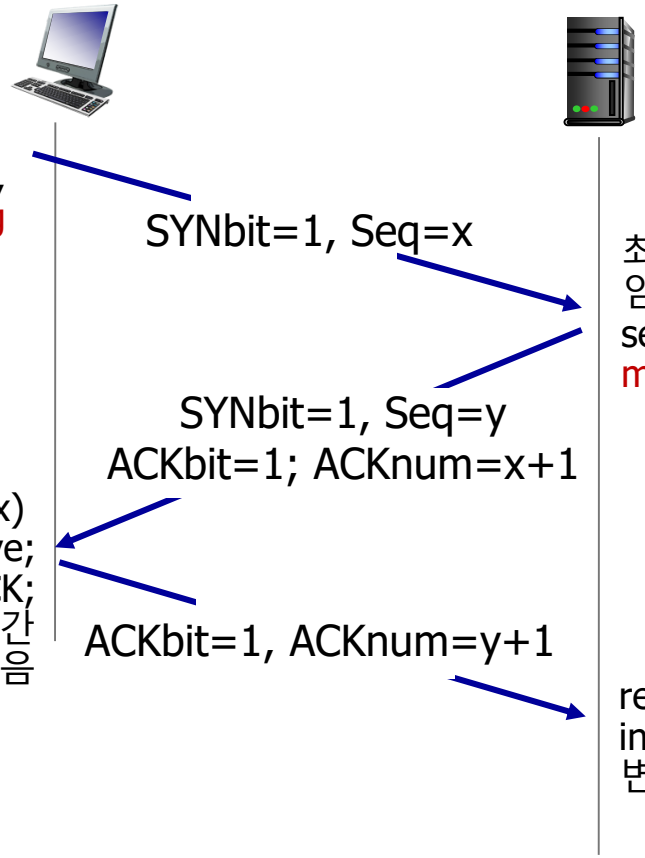
TCP 3-way handshake (세 방향 핸드셰이크)

Server state

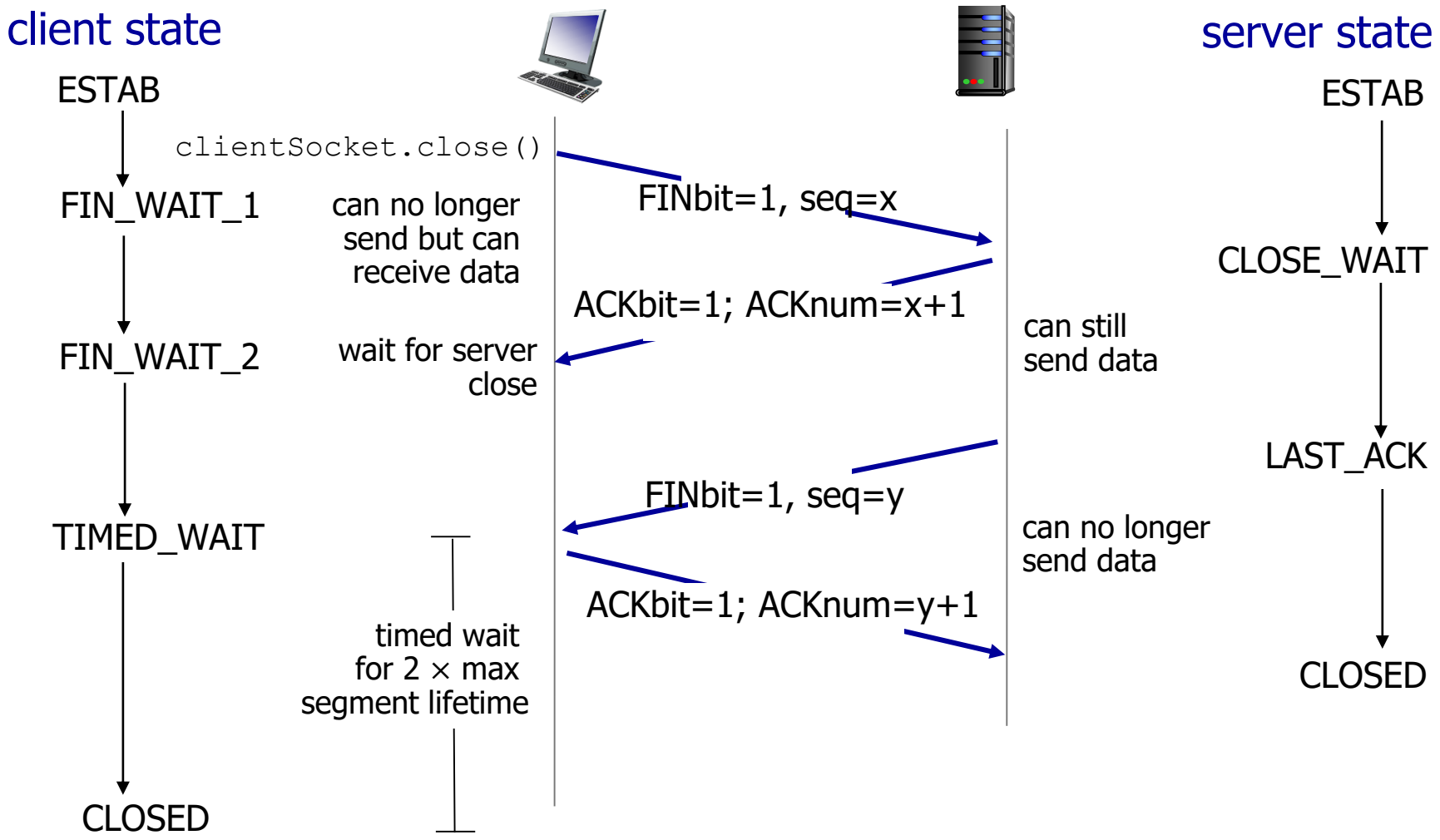
```
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
connectionSocket, addr = serverSocket.accept()
```

Client state

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

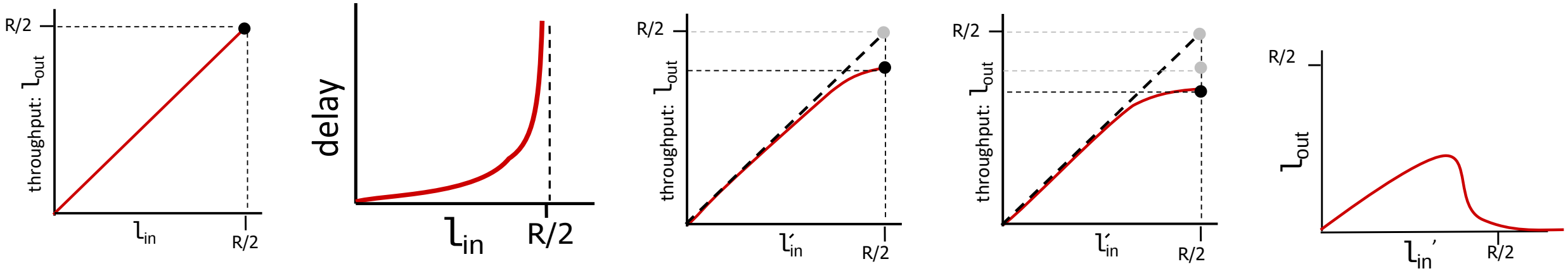


Closing a TCP connection (4-way handshake)



혼잡의 원인과 비용: insights

- 처리율은 링크 용량을 초과할 수 없음
- 처리율이 링크 용량에 가까워질수록 큐잉 지연 시간이 급격히 증가
- 패킷 손실을 보상하기 위한 재전송은 유효 처리율 감소시킴
- 긴 지연으로 발생하는 불필요한 재전송은 유효 처리율을 더욱 감소시킴
- 경로상 라우터에서 패킷이 손실되면, 출발지 호스트부터 해당 라우터까지 패킷 전송에서 사용된 모든 자원 (링크 용량과 버퍼링)이 낭비됨



혼잡 제어에 대한 접근 방식

종단 간의 혼잡 제어 (end-to-end congestion control):

- 네트워크에서 명시적인 피드백 없음
- 송신자는 패킷 손실 및 지연 시간을 기반으로 혼잡 발생 여부를 추론함
- TCP가 선택한 접근 방식

네트워크 지원 혼잡 제어 (Network-assisted congestion control):

- 혼잡이 발생한 라우터는 송/수신 호스트에 직접적인 피드백을 제공
- 혼잡 수준을 표시하거나 현재 출력 링크 전송률을 명시적으로 알림
- 혼잡 정보 피드백 방식
  - 송신자에게 직접 피드백: 초크 (choke) 패킷 활용
  - 수신자를 경유하는 피드백: RTT 시간 발생
- TCP에서 선택적으로 구현

TCP 혼잡 제어: AIMD

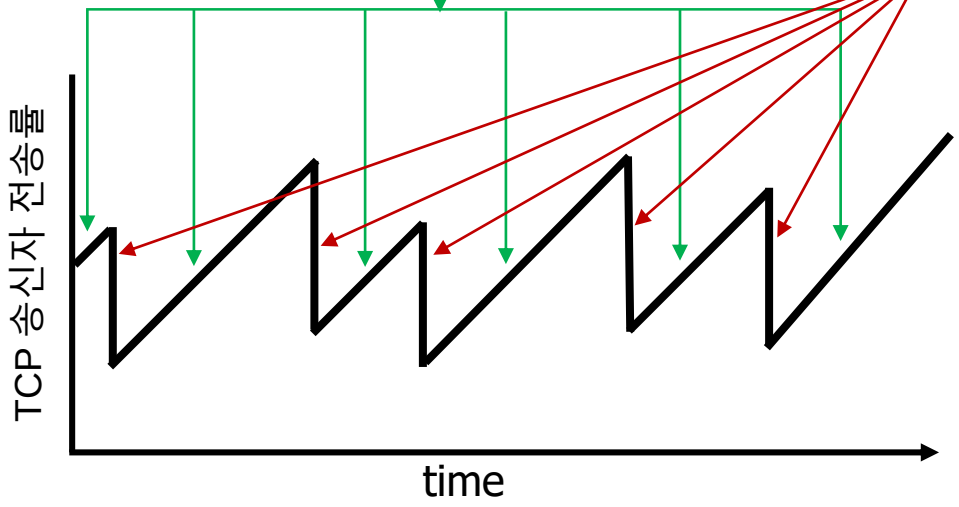
- 접근법: 송신자는 패킷 손실 (혼잡)이 발생할 때까지 전송 속도를 높이고, 손실 발생 시 전송률을 낮춤

가법적 증가 (*Additive Increase*)

손실이 감지될 때까지, RTT마다 최대 세그먼트 크기 '하나' 씩 전송률을 증가

승법적 감소 (*Multiplicative Decrease*)

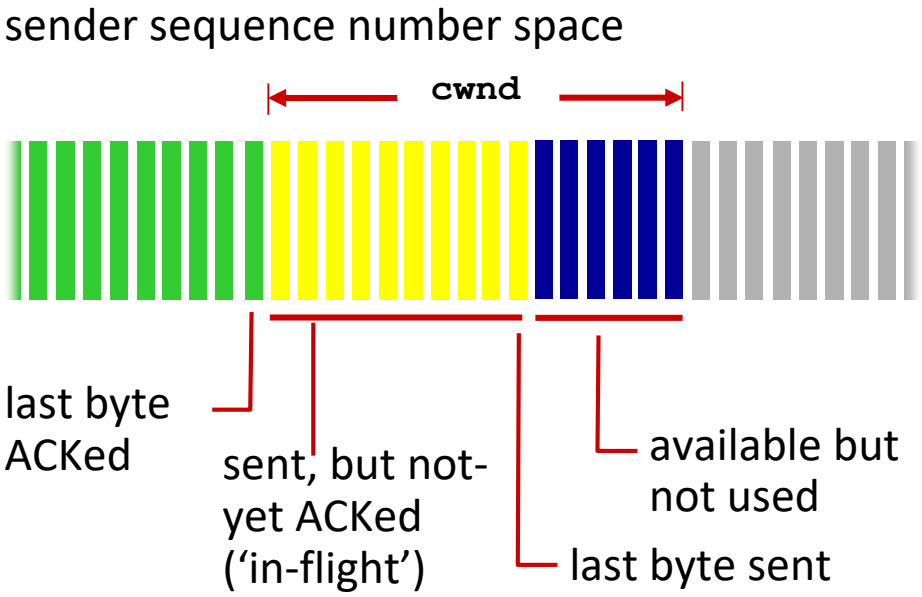
손실 감지되면 전송률을 '절반'으로 줄임



AIMD의 톱니 동작:

- 대역폭 탐색

TCP 혼잡 제어: 상세



$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{cwnd}$$

윈도우 사이즈: $N = min\ (rwnd,\ cwnd)$

$$\text{TCP 전송률} \approx \frac{\text{cwnd}}{\text{RTT}}\ \text{bytes/sec}$$

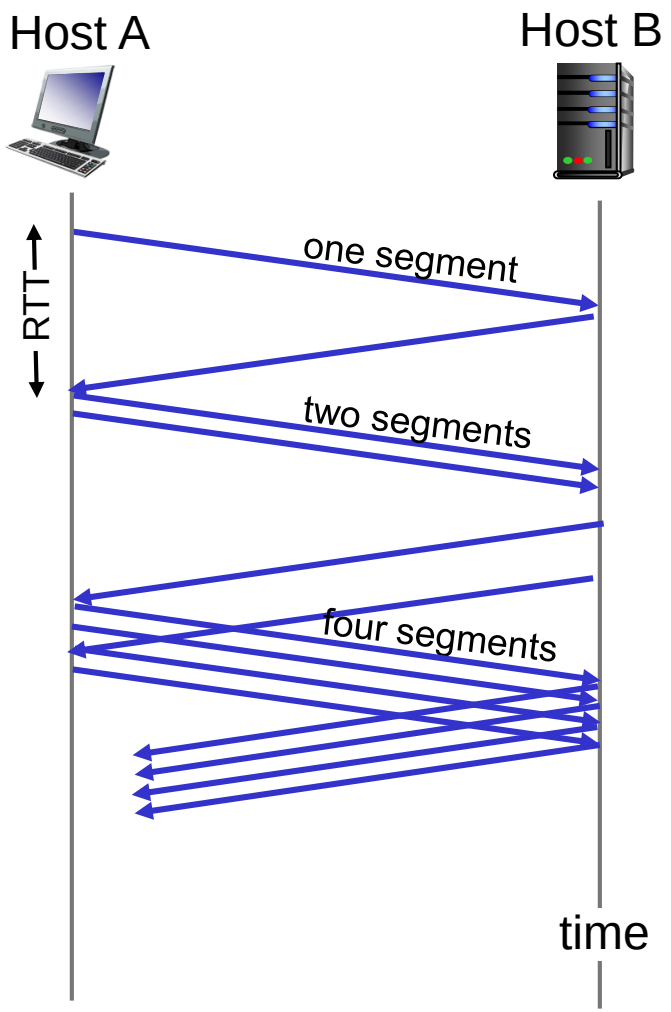
- 혼잡 윈도우 (congestion window, cwnd)는 TCP 송신자는 전송을 제한
- cwnd는 탐지된 네트워크 혼잡에 따라 동적으로 조정됨 (TCP의 혼잡 제어 구현)

TCP의 대략적인 전송 동작:

- cwnd 만큼 바이트를 보내고, 확인응답을 위해 RTT 만큼 기다린 다음, cwnd 값을 조절하면서 더 많은 바이트를 전송

TCP 슬로 스타트 (slow start)

- TCP 연결을 시작했거나 타임 아웃이 발생했을 때 '슬로 스타트' 시작
  - **cwnd** = 1 MSS로 초기화
- 이후 매 RTT마다 **cwnd** 값은 두배 씩 증가
  - ACK가 수신될 때 마다, cwnd = cwnd + MSS
  - **[예]** 현재 cwnd = 4이면, 4개의 세그먼트를 전송하고 RTT 후 4개의 ACK를 성공적으로 수신한다면, 다음 cwnd: 현재 cwnd (4) + ACK (4) = 8
- 요약: 초기 속도는 느리지만, 기하급수적 (두배 씩)으로 빠르게 증가!



슬로 스타트에서 혼잡 회피로 전환

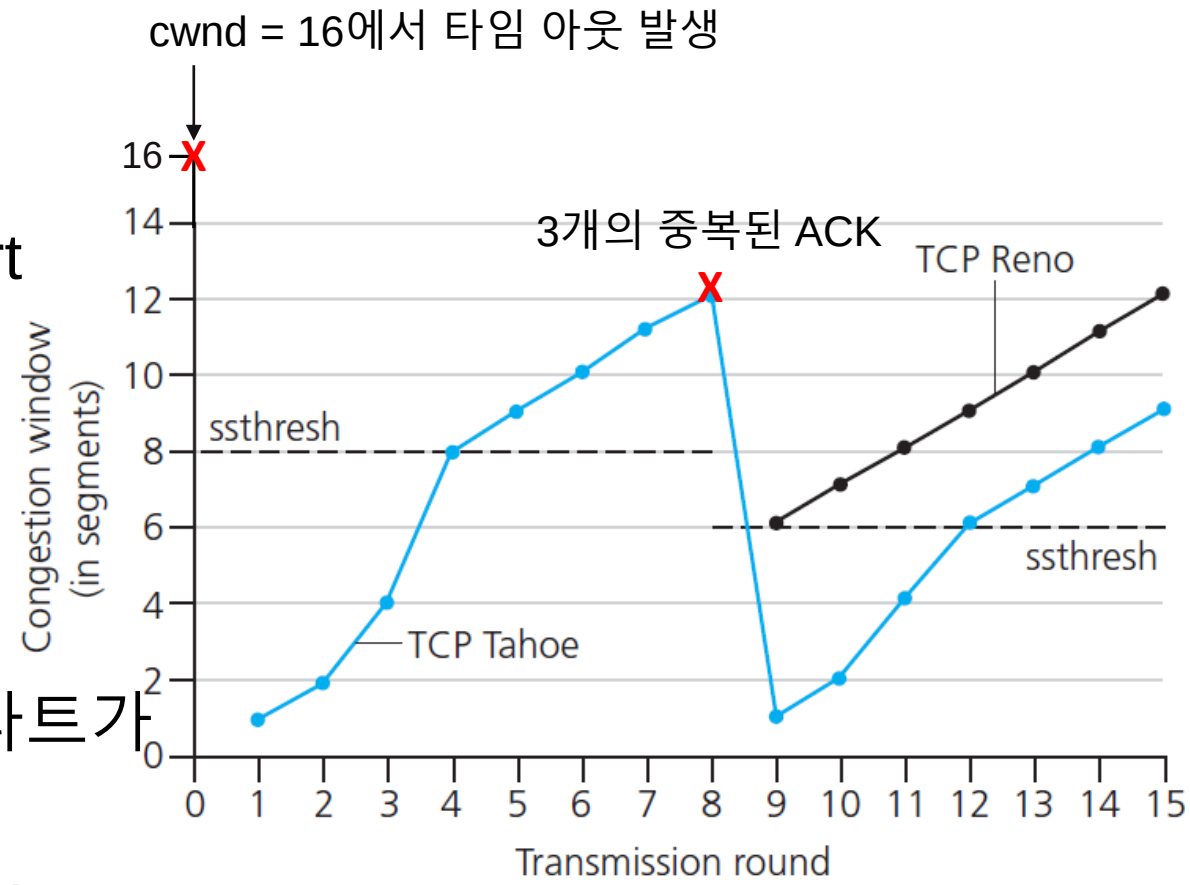
Q: 언제까지 기하급수적으로 증가할 수 있을까?

A: cwnd가 슬로 스타트의 임계 값 (slow start threshold, ssthresh)에 도달할 때까지

- ssthresh: 혼잡이 마지막으로 탐지된 시점에서의 cwnd 값의 반

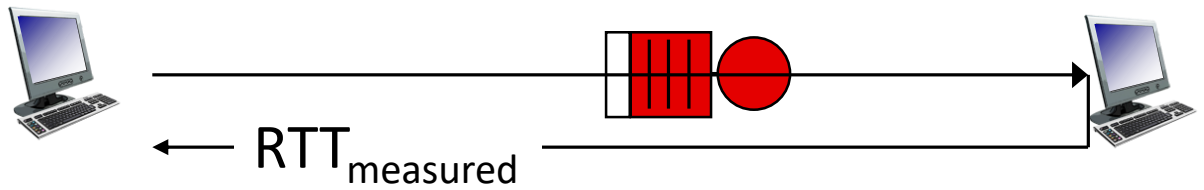
혼잡 회피 (congestion avoidance)

- cwnd 값이 ssthresh에 도달하면, 슬로 스타트가 종료되고 혼잡 회피 시작
- '전체' cwnd에 대한 확인응답이 되면, cwnd 값을 하나씩 증가 (선형적으로 증가)



지연 기반 (Delay-based) 혼잡 제어

송신자와 수신자 간의 파이프를 '충분히 채우되, 꽉 채우지 말자': 병목 링크를 전송 중인 상태로 계속 유지 (처리율의 최대화), 하지만 긴 지연 시간과 버퍼링은 피해야 됨 (지연을 낮게 유지하면서)



$$\text{처리율 측정} = \frac{\text{마지막 RTT 간격에 전송된 바이트 수}}{RTT_{measured}}$$

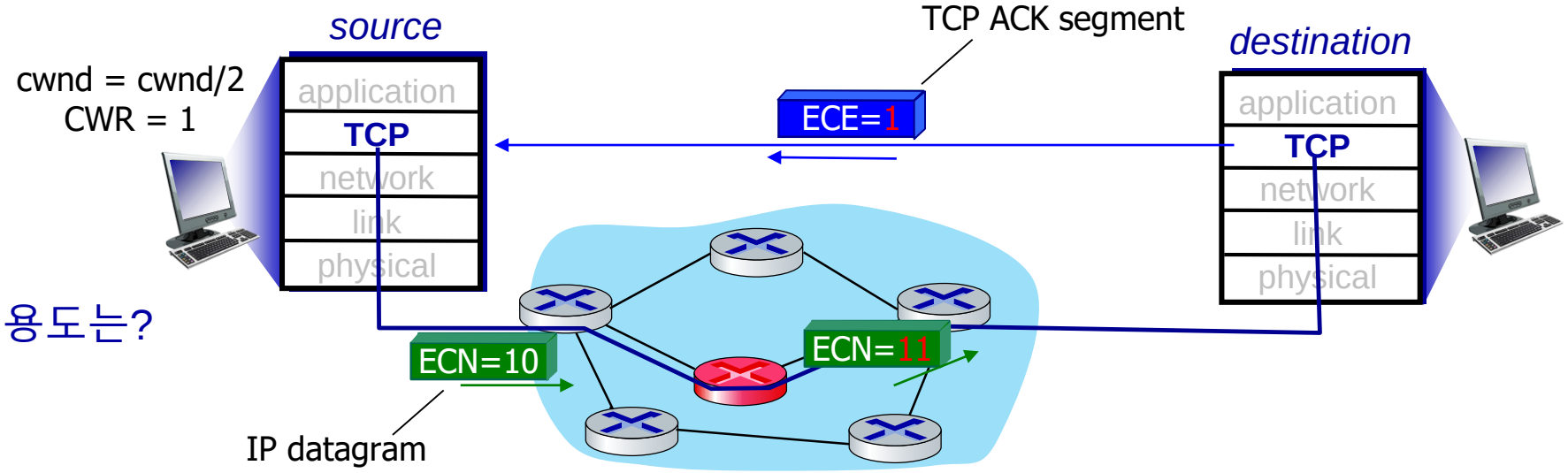
지연 기반 혼잡 제어 (TCP vegas, 1995): 손실을 유발하지 않고, 혼잡을 제어

- RTT_{min} : 송신자가 측정한 RTT의 최솟값 (혼잡하지 않고, 패킷의 최소 대기 시간)
- 혼잡하지 않는 처리율은 cwnd/RTT_{min}
- 만약 측정된 처리율이 혼잡하지 않은 처리율에 '매우 근접'하면, 혼잡이 발생하지 않았다고 판단하고 cwnd를 선형적으로 증가
- 그렇지 않고, 혼잡하지 않은 처리율보다 '훨씬 아래'로 측정되면, 혼잡이 발생했다고 판단하고 cwnd를 선형적으로 감소

명시적 혼잡 알림 (Explicit Congestion Notification, ECN)

최근 배포된 TCP 일부는 네트워크 지원 혼잡 제어를 구현함:

- 혼잡이 발생한 라우터는 IP 헤더의 2비트 (type of service 필드의 ECN 비트)를 통해 혼잡 발생을 표시
  - 라우터의 혼잡 기준은 라우터 공급업체 및 네트워크 운영자가 결정
- 혼잡 표시가 수신자에게 전달되며, 수신자는 ACK 세그먼트 헤더의 ECE 비트를 1로 설정하여 송신자에게 혼잡이 발생 했음을 알림
- involves both IP (IP header ECN bit marking) and TCP (TCP header CWR, ECE bit marking)



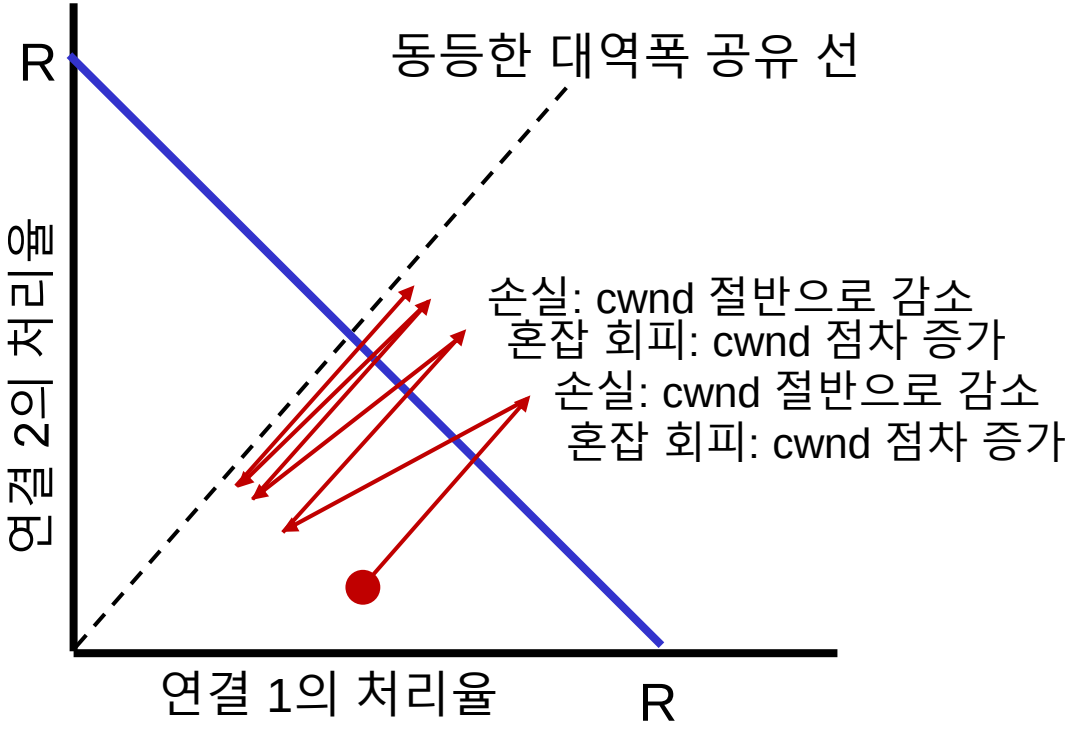
CWR bit의 용도는?

ECE: Explicit Congestion notification Echo, CWR: Congestion Window Reduced

Q: 과연 TCP는 공평한가?

[예] 두개의 경쟁 TCP 연결:

- 가법적 증가 (additive increase)는 기울기가 1 (45°)이고 전체적으로 처리율을 증가시킴
- 승법적 감소 (multiplicative decrease)는 처리율을 비례적(절반)으로 감소시킴



Is TCP fair?

A: 이상적인 시나리오에서, TCP는 공평함:

- RTT 및 MSS 동일
- 연결 숫자는 고정이며, 언제나 혼잡 회피 방식으로만 동작

모든 네트워크 애플리케이션이 '공평' 할까?

공평성과 UDP

- 많은 멀티미디어 애플리케이션에서 TCP를 사용하지 않음
  - 혼잡 제어로 전송률이 조절되는 것을 원하지 않음
- 대신 UDP를 사용:
  - 일정한 속도로 오디오/비디오 전송이 가능하며, 패킷 손실을 허용
- TCP 관점에서 UDP는 공평하지 않음

공평성과 병렬 TCP 연결

- 애플리케이션이 두 호스트 간에 여러 병렬 TCP 연결을 설정할 수 있음
- [예] 9개의 클라이언트/서버 응용이 TCP 연결을 하나씩 설정하여 전송률 R의 링크를 공유하고 있음
  - 새로운 응용이 TCP 연결을 1개를 설정하고 R/10 전송률을 가짐
  - 새로운 응용이 TCP 연결을 병렬적으로 9개를 설정하여 R/2 전송률을 가짐

Q & A